

University of Innsbruck

Department of Computer Science

Distributed and Parallel Systems Group



Faculty of Mathematics,
Computer Science and Physics

Visualizing Celerity Traces in VR

by

Lorenz R. Smidt, BSc.

Master's Thesis

Supervisor: Dr. Peter Thoman

March 18, 2025

Declaration

By my own signature I declare that I produced this work as the sole author, working independently, and that I did not use any sources and aids other than those referenced in the text. All passages borrowed from external sources, verbatim or by content, are explicitly identified as such.

Signature:

Date: 18.03.2025

Contents

1	Introduction	1
2	Related Work	3
2.1	The Celerity Project	3
2.2	Execution Traces	4
2.3	Visualization in Virtual Reality	5
2.4	Virtual Reality Design Challenges	10
3	Background	13
3.1	Celerity's Structure	13
3.2	Virtual Reality Technology	14
3.3	Unity Engine	15
3.4	OpenXR and Unity	16
4	Implementation	19
4.1	Overview	20
4.2	Trace Data	20
4.2.1	Trace Data Format	21
4.2.2	Pre-Processing	22
4.2.3	Internal Data Structure Design	24
4.3	Visualization	26
4.3.1	Commands	26
4.3.2	Command Details	29
4.3.3	Dependencies	30
4.3.4	Auxiliary Information	34
4.3.5	Display Modes	35
4.4	Virtual Reality Interface	39
4.4.1	Scene Movement and Interaction	39
4.4.2	Wrist Menu	43
5	Results	49
5.1	Usability	49
5.2	Comparison to 2D Solution	50
5.3	Performance	53
5.3.1	Execution Times of Selected Parts	53
5.3.2	Runtime Frames Per Second	56
5.3.3	Future Optimizations	57
6	Conclusion and Future Work	59

Contents

Appendix	61
1 Wrist Menu Sections	61
2 Display Mode Settings Interfaces	62
3 Hardware and Software Used for Testing	64
4 Additional Performance Measurement Results	65

Abstract

The increasing availability of Virtual Reality (VR) hardware in the consumer space opens the door for novel application areas to take advantage of its unique properties. Its ability to simulate virtual three-dimensional space presents an opportunity for the field of data virtualization. Especially in cases where the data is too complex to effectively visualize on traditional monitors this added dimension has the potential to dramatically increase visual clarity. One source of such complex data is the Celerity project[13][14][10]. Its goal is to automatically distribute a given program across a cluster of distributed memory compute nodes. Due to the challenges presented by this task, its runtime behavior tends to be quite complex in nature. In order to analyze its performance an execution trace can be recorded during its runtime, allowing the user to inspect its inner workings after the fact. In this thesis a VR application for visualizing these Celerity execution traces is implemented, showcasing the potential advantages the hardware platform offers when compared to traditional solutions.

1 Introduction

The Celerity project[13][14][10] aims to automatically distribute a given program across a cluster of compute units. This type of work is generally done manually due to the high complexity in managing the various dependencies that arise from the distribution process. For example, if a computation is split across two compute nodes (with separate memory) and one part of the computation requires the results from the other one, the data has to be manually transferred between the two nodes. Managing these types of inter-node dependencies leads to an increasingly high implementation and maintenance complexity in distributed programs. By providing a deterministic and, most importantly, automatic way to handle these challenges, the Celerity project aims to greatly reduce the workload on the user when implementing these kinds of programs.

However, this automatic distribution process is in itself highly complex. In order to assess the performance and correctness of the produced distribution and its handling of dependencies, a so-called execution trace can be recorded while Celerity is running. Execution traces are a common tool for analyzing the execution of complex programs in general. They typically contain all of the information necessary to reconstruct a programs behavior at runtime, allowing the user to manually step through the execution and identify potential issues. Typically a separate trace visualization tool is necessary in order to display the trace data to the user. A variety of these tools is available, each with their own advantages, drawbacks and features. So far the vast majority of those target traditional two-dimensional (2D) display technologies, i.e. flat monitors.

Technology for displaying three-dimensional (3D) content to users has reached the consumer market over the last two decades in the form of various [Virtual Reality \(VR\)](#) and [Augmented Reality \(AR\)](#) devices. While early models served more as a proof-of-concept, current hardware by various manufacturers is generally advanced enough for use in complex applications. However, the user interfaces required by these platforms lead to new design challenges, some of which are still open to debate (see section 2.4).

One potential application area for [VR](#) is the aforementioned execution trace visualization. Due to its ability to simulate 3D spaces, it is possible to spread out the increasingly complex data contained in trace files over a large area of virtual space. This has the potential to increase visual clarity and enable novel data layout methods which would otherwise be impractical for traditional flat displays.

As such, the [VR](#) trace visualizer developed in this thesis aims to take advantage of the opportunities offered by the [VR](#) hardware platform for the task of visualizing Celerity execution traces. It features a variety of purpose-built visualization aides as well as two trace layout variants. As of this writing the visualizer is the first of its kind, although similar projects in other application areas have been created in the past.

Chapters 2 and 3 cover previous work in related fields as well as the technical foundations on which this application builds. The main design and implementation of the trace visualizer is described in chapter 4, starting with the raw trace data, building up the visualization components, and ending with the user interface. Chapter 5 provides an assessment of the programs overall usability, performance, and how it compares to state-of-the art 2D trace visualizers. And finally, chapter 6 summarizes the results and provides a future outlook.

2 Related Work

While there does not exist a project similar to this one as of its writing, a variety of work has been done in the past on adjacent topics of research. This chapter focuses on the scientific work used as a foundation and/or inspiration for this thesis, while the next chapter covers some of the technical background required for its implementation.

2.1 The Celerity Project

The basis for this thesis is the Celerity project. It was first described by Thoman et al. in 2018 [13] and has since received various improvements, such as the ones detailed by Thoman et al. [14] in 2019, and Salzmann et al. [10] in 2023. The project aims to provide an easy-to-use interface for targeting distributed memory compute clusters (in reality mostly GPU clusters) and offers automatic work and data distribution in a [Single Program Multiple Data \(SPMD\)](#) fashion. It achieves this by extending the existing SYCL¹ domain-specific embedded language with its own API, while only requiring minimal changes to the code to complete the migration. Furthermore, the system is designed to maintain performance similar to alternative implementation methods for targeting distributed memory compute clusters, such as MPI+SYCL or MPI+OpenCL.

At its core the project combines SYCL programs with additional information provided by the user and distributes it over a compute cluster using MPI. SYCL programs generally consist of one or more *compute kernels* performing operations on *buffers*. These kernels may have dependencies between each other as dictated by their access patterns to the corresponding buffers. To enable automatic distribution of a SYCL program, so-called *range mappers*, functions describing buffer access patterns of compute kernels, are needed. With these in hand the system can automatically distribute tasks as well as the required data across the cluster and combine the results afterwards. Each part of the cluster is called a *compute node* or *worker node*, encapsulating all compute device (e.g. GPUs) of one machine. Since distribution of work can (and often does) lead to dependencies between parts of the job, the system needs to coordinate and keep track of these at runtime and coordinate the compute nodes accordingly.

The original implementation of Celerity [13] used a centralized execution model where a single master node was responsible for the entire distribution process. However, since this approach turned out to be a bottleneck when scaling to larger clusters, it was later improved to a distributed management model [10]. While both implementations provide comparable performance compared to alternative programming models in various applications, only the later version achieves near-linear scaling on larger GPU clusters. The [VR](#) trace visualizer implemented in this thesis is built to only work with the earlier version of the Celerity runtime and the trace files it produces since work began before the new version was released.

¹<https://www.khronos.org/sycl/>, accessed 01.03.2025

2.2 Execution Traces

Trying to analyze the runtime performance of complex programs, either manually written or semi-automatically generated like a Celerity execution run, is challenging. The tool of choice for this task is tracing, of which various forms exist. However, in the end the recorded information (i.e. the trace) has to be visualized in a way such that the user can easily navigate and interpret the data.

A Controlled Experiment for Program Comprehension through Trace Visualization

Cornelissen et al. [5] conducted the first quantitative experiment to measure the utility of execution trace visualization for program comprehension tasks in 2011. 34 volunteers were enlisted for help, coming from different backgrounds including multiple PhD candidates, MSc students, postdocs, professors, and participants from industry. The experiment was centered around the *CHECKSTYLE* tool, for which participants had to answer a number of comprehension questions. As a baseline the users were given a traditional [Integrated Development Environment \(IDE\)](#), in this case Eclipse. The authors set out to measure the effectiveness of trace visualization by observing the increase in users' performance when also given access to their own trace visualization software *EXTRAVIS*. Participants in the group with the trace viewer were given two execution traces for different input sizes, while both groups had access to the source code. In order to reduce the impact of prior knowledge about the type of tasks, the participants were each ranked in five different areas in expertise. The test and control groups were then manually assembled with the goal of balancing out the average expertise of the participants in each group. The users were presented with 8 typical source code comprehension tasks and had their performance as well as their time taken recorded. Additionally a debriefing questionnaire was provided.

Evaluating the results showed that the availability of *EXTRAVIS* with the provided traces showed significant improvements in both time taken and correctness. On average, the time taken was reduced by 22% per task and the correctness of the results increased by 43%. Out of the eight tasks, the tracing tool was used on average for seven of those and "the tool was actually found useful in six of those tasks (86 percent)" by the users. The authors do note that due to the experiment size no concrete conclusions can be drawn as to the areas in which tracing is the most useful. However, it does provide strong evidence that tracing tools are useful for tasks requiring the understanding of a system's global behaviour and structural relations. They can also help when assessing contexts in which many parts of a program interact with each other since a trace can capture the active parts all in one place.

Visualization of Distributed Program Execution

Distributed programs run on multiple machines at the same time. Creating execution traces of such programs usually results in multiple independent traces being created, one for each machine performing computations. However, when trying to understand the operation of a distributed program or trying to find issues it is helpful to consider the interactions between the involved machines as well as the overall environment. To this end Satwik et al. [11] extended the Java Interactive Visualization Environment (JIVE) to combine the traces from all the machines involved in a program into one. The important issue to be solved is to keep all events in the correct causal order when merging the traces. Using JIVE the authors generate sequence diagrams for the executions on each of the machines. In this case each event refers to a transition on a sequence diagram, and the end goal is to create a sequence diagram containing the events from all machines.

The merging process starts by generating unique integer identifiers for each event across all diagrams. Then the events for communication, i.e. sends and receives, are separated out. With these separated they can be matched by with the corresponding events from different machines using included message ids. As a final step all events are combined into a unified sequence diagram in which the individual send/receive events are replaced with complete transfer events between machines. According to the authors the result "provides a complete history of the execution as it happened on multiple systems." This representation is potentially useful in identifying race conditions and deadlocks.

2.3 Visualization in Virtual Reality

Ever since [VR](#) technology has become more accessible over the last few years, researchers have been looking for ways to use it effectively in scientific applications. One key area is the visualization of data in [VR](#).

Visualization in Virtual Reality: a Systematic Review

To this end, Korkut and Surer [7] conducted a systematic literature review in 2023 to identify the current state-of-the-art in the topic, as well as future directions and common challenges when working with this technology. According to the authors "[...] VR facilitates the manipulation and analysis of data by using the advantages of 3D environments". It can also allow for the design of exploration-based environments which encourage active participation instead of passive observation. This makes it an effective tool in learning and training environments, but also allows scientific data to be visualized in unique ways. The authors argue that [VR](#) visualizations should incorporate interactivity to allow the user to explore the represented data. In addition multiple perspectives can be used to present the data with the intention to reveal hidden features that may be hidden when using other mediums for visualization.

However, when compared to conventional 2D desktops, "immersive visualization can be considerably different, forcing scientists to reconsider how they approach the data in some cases." Directly converting 2D visualization techniques to a 3D representation can also cause clarity issues. Visual phenomena that are only present in 3D space, "such as occlusion and perspective distortion, may lead to wrong interpretation in analytical use cases." Furthermore, [VR](#) has comparatively high performance requirements in order for a user experience to be deemed acceptable. Improper implementation of headset tracking or user locomotion as well as low frame rates can potentially lead to motion sickness, also called cybersickness in this context.

The authors also note that the visualization techniques used in various domains are closely related to each other. Techniques used to solve problems in one area may be used to solve similar issues in other areas in the same way since they all face similar technical challenges. One technique of note is the use of additional features such as textures, colors, or shapes to convey additional information. These can be seen as an extension of an object's position in space and allow multidimensional data (e.g. different object types or properties) to be visualized. However, it is also pointed out that "a good visualization should protect the balance between esthetics [sic] and functionality." In the worst-case scenario an improper visualization may lead to the user misunderstanding the data and drawing wrong conclusions.

Korkut and Surer [7] state that the visualization field in general uses visualization techniques that tend to be firmly rooted, but not necessarily proven. The same is even more true for [VR](#) visualization, where in addition to a lack of established visualization techniques, the user interface is also mostly based on popular implementations instead of thorough research. This is further complicated by advancements in [VR](#) hardware enabling the use of new interaction methods stemming from technologies such as eye tracking. As identified

2 Related Work

by Korkut and Surer [7], "within visualization, the most mature research areas have come from training, architecture, and game technologies." But there has been good progress in other areas as well, such as in the field of explainable artificial intelligence or neurological research.

Virtual and Augmented Reality Applications to Support Data Analysis and Assessment of Science and Engineering

Su et al. [12] outline in a 2020 paper how scientists at the Combat Capabilities Development Command Army Research Laboratory use [VR](#) and [AR](#) technologies to support their research. According to them there are four critical factors concerning the implementation of data analysis and assessment workflows: "functionality of the software; performance of the software; ease of hardware setup; and ease of software setup." The authors identify two ways of adding [VR](#) or [AR](#) support to data analysis programs and workflows: Either by adding [VR/AR](#) functionality into an existing program, or by re-implementing the program's features in a [VR/AR](#) application. Both methods come with drawbacks: Adding [VR/AR](#) functionality requires a rendering pipeline that is capable of producing stereoscopic images in real time, as well as a way of handling the tracking data. For obvious reasons most data analysis tools do not incorporate that functionality by default, and adding it retroactively tends to lead to performance issues. Re-implementing data analysis features inside a [VR/AR](#) application has, besides the obvious drawback of having to re-create the tools, no guarantee of actually resulting in good performance either. As the authors note in one example, actually showing the data at high enough of a resolution such that interesting details are visible "reduced the interactivity of a VR application to a slow slide show." And in another example only a subset of the corresponding application's features were implemented in the [VR](#) version of the program, limiting its utility.

One example that Su et al. [12] highlight in their work is an application intended to help engineers uncover and analyse complex 3D features. Their basis is an existing workflow based on simulations performed on [High Performance Computing \(HPC\)](#) resources, the results of which are then loaded into a visualization tool which generates an animation from specific time steps in the simulation. One of the issues with this existing approach is the fixed viewpoint: If the scientist wants to view the simulation from a different perspective the animation has to first be re-rendered by the system. To improve this they extended their existing workflow with an [AR](#)-style interface called zSpace, which they classify as a "semiimmersive desktop VR device" based on polarized stereo glasses and a specialized screen. This allows the use of traditional mouse and keyboard interfaces as well as quick switching between 3D and 2D visualizations using different monitors. As a further development they implemented a full [VR](#) visualizer using the Unity game engine with the OpenVR plugin, which they use in conjunction with an HTC Vive [Head-Mounted Display \(HMD\)](#). A new set of interaction gestures is implemented to go along with the included [VR](#) controllers: Pressing the trigger button on either hand allows the user to grab the model and move it around. Holding both triggers at the same time scales the model up or down, similar to the two-finger scaling on a touch device. Additionally, pressing the thumbpad on the controller allows the user to move in the direction in which they are pointing. In order to overcome the high performance requirements for a [VR](#) application of this scale, the authors made use of existing [HPC](#) resources. The [HPC](#) machine is used to process and store the data, while the actual client streams the data from the [HPC](#) machine and renders it locally. This gives the user a usable [VR](#) experience and the "ability to analyze large scientific data." Overall the authors conclude that the use of both 2D and 3D techniques each have their own benefits in helping the user understand scientific data.

A Comparative Evaluation of the Effectiveness of Virtual Reality, 3D Visualization and 2D Visual Interactive Simulation: An Exploratory Meta-Analysis

Another application area for VR visualization, or in more general terms interactive 3D visualization, is the field of **Discrete-Event Simulation (DES)**. Akpan and Shanker [1] conducted a meta-analysis in 2019 of research in that domain from the years 2000-2016, with the goal of identifying key benefits of VR/3D visualization techniques. They note that at the time 3D displays were referred to as VR by simulation vendors and users, but in reality these technologies differ from fully-immersive VR HMDs. Despite this fact the authors decided to include both technologies under the umbrella term of VR, and as such analyzed research results using either one.

Two of the main points that are assessed is whether VR visualizations enhance user performance and are faster to use compared to 2D ones when performing DES tasks. The DES domain spans various tasks with different focuses and therefore has a range of user requirements to the tools used. As shown in table 1 most DES tasks show clear benefits in both user performance as well as time taken to complete when using VR technologies. This trend stays mostly consistent "across the different problem domains, problems tackled, and the research methods, irrespective of the year in which the study was conducted." Only the task of model development is more heavily weighted towards 2D tools, with 90% of relevant studies stating that 2D tools are faster. The authors also note that the majority of the relevant studies, even the ones favoring VR, "raised a concern that developing a DES model remains more difficult and takes longer" using VR technologies. Contrarily, the tasks of model validation and verification show a clear benefit of VR since it allows domain experts to quickly and easily identify issues in the developed model. Especially mistakes caused by the use of incorrect data can be identified using 3D models as it allows the experts to easily spot if the model's behaviour appears unusual compared to real life.

DES activities	Which display enhances performance?				Performance time			
	n	3D better	2D better	Same	n	3D shorter	2D shorter	Same
Model development	22	13	6	3	10	1	9	-
Experimentation	17	16	1	-	-	-	-	-
Model run	9	7	2	-	-	-	-	-
Verification	19	18	-	1	5	5	-	-
Validation	26	24	-	2	6	6	-	-
Analysis	28	28	-	-	-	-	-	-
Presentation	32	32	-	-	-	-	-	-
Problem definition	5	3	1	1	1	1	-	-
Conceptual modeling	4	-	2	2	-	-	-	-
Implementation	1	-	-	1	-	-	-	-
n = 162								

Table 1: Summary of results from a comparison of papers on the impacts of 3D/VR vs 2D displays on **Discrete-Event Simulation (DES)** tasks compiled by Akpan and Shanker [1]. Note that performance in this context refers to a users' ability to perform a task. In most tasks the 3D/VR display shows a clear improvement over 2D counterparts.

On the use of Virtual Reality in Software Visualization: The Case of the City Metaphor

Another effective use-case of **VR** visualization in the field of software visualization was shown by Romano et al. [9] in 2019. The authors evaluated the effectiveness of the city metaphor when extended from 3D visualizations on 2D monitors to immersive **VR**. They also compared the results of both user performance as well as time taken to an **IDE** (in this case Eclipse) with plug-ins for collecting code metrics. In the context of software visualization, the city metaphor is a method of representing code from object-oriented languages in the shape of a city. Each building represents a class, with its height being the number of methods within it, the base size being the number of attributes, and the building's color being a gradient based on the lines of code within the class. These buildings are then grouped into blocks based on packages. This representation aims to make it easier for the user to differentiate types of classes, such as god or data classes. An example of this is shown in figure 1.

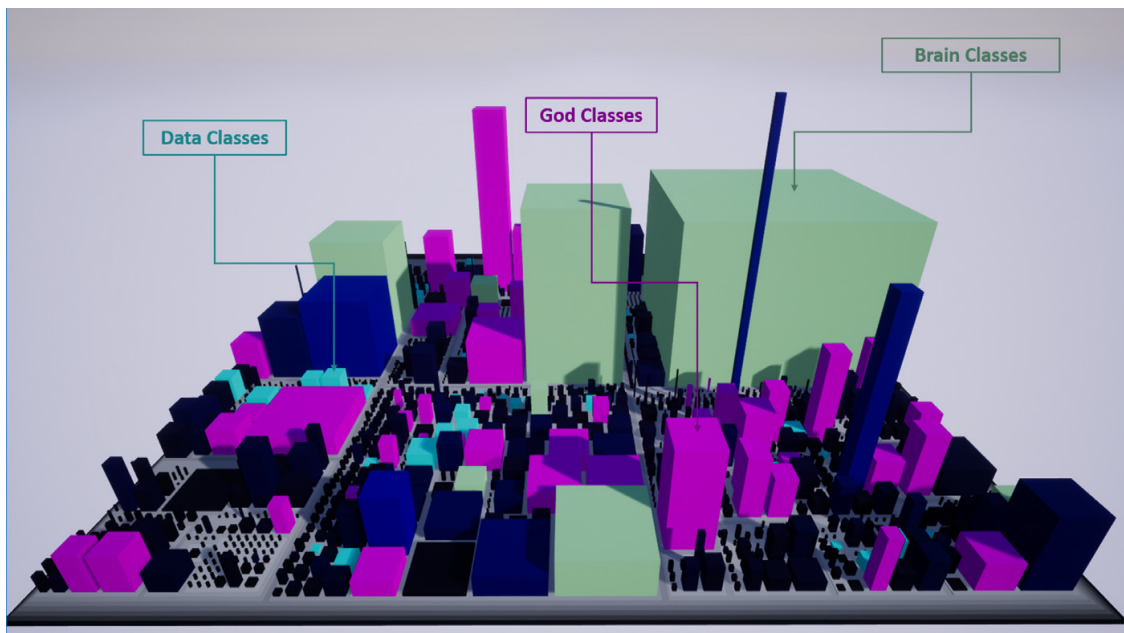


Figure 1: An example of the city metaphor being used to visualize software. The representation of individual classes as buildings with different properties help in identifying different types of classes. Taken from Romano et al. [9].

The authors enlisted the help of 42 volunteer subjects for their evaluation and asked them to perform a subset of the FindBugs² tasks. The participants were also asked to fill out questionnaires both before and after performing the tasks. Each of the tools was configured to show the same information to the participants. The results show that the city metaphor improves correctness in program comprehension tasks over the traditional **IDE** environment. Notably both types of city metaphor visualizations, i.e. 3D on a flat screen or immersive **VR**, result in comparable performance. However, the **VR** environment does significantly decrease the time spent to accomplish the assigned tasks when compared to both non-**VR** tools. Analyzing the questionnaires shows that the participants consider the **VR** city metaphor "more useful" compared to the non-**VR** version, which in turn is again considered "more useful" compared to the **IDE**. Furthermore, the authors note that "the use of the immersive virtual reality medium also leads to higher satisfaction of the participants." [9] One

²Wettel et al., [http://refhub.elsevier.com/S0950-5849\(19\)30140-5/sbref0036](http://refhub.elsevier.com/S0950-5849(19)30140-5/sbref0036), 2010

drawback of the VR environment is the possibility of side effects. The authors note that "headaches, nausea, and visual annoyance" can be induced by the use of the HMD used, in their case the Oculus Rift.

A Virtual Reality Visualization Tool for Neuron Tracing

In a 2018 paper Usher et al. [15] developed a VR tool to help neuroscientists with the task of tracing neurons in brain scans. Due to the nature of the problem and with only few automated tools being available, this work is usually done manually. In the traditional tools multiple 2D scans are stacked on top of each other and the user scrolls through them, marking the path of neurons along the way. However, this allows for very limited spatial perception, since much of the volumetric information is only implied. According to the authors this shortcoming is one of the contributing factors for tracing errors. They aim to improve this workflow by visualizing the scanned data in 3D and implementing an interface for navigating and interacting with it in VR. One major problem with this approach, which is also faced by 2D tools, is the size of the input data. According to the authors "datasets can range from hundreds of gigabytes to terabytes", which exceeds the RAM and VRAM of virtually all workstations. Their solution is to implement a multi-stage paging system that dynamically streams the data from the hard drive to the RAM, and then further to the GPU as needed. The system is designed for the HTC Vive, at the time one of the best consumer VR HMDs. In order to give experts the easiest barrier to entry, a lot of the controls are designed to resemble their 2D counterparts. To evaluate the effectiveness of the tool, 3 novice users and 4 experts used both the industry-standard 2D tool NeuroLucida and the new 3D VR tool to work on two reference datasets. Comparing their performance in both reveals a similar or slightly higher level of accuracy using the VR tool while being faster on average. Participants also reported that the tool is more intuitive to use than the 2D counterpart, while also being "easier to interact with, and less fatiguing."

Three years later in 2021, McDonald together with many of the original authors [8] improved the tool developed by Usher et al. [15] further. They added a semi-automatic tracing extension which guides the users while tracing neurons. This addition allows the user to more accurately follow the path of the neurons while still being able to make individual decisions. Furthermore, a so-called flashlight feature was added, which shows an overview of the automatically predicted neuron paths in a given area. Testers reported that this feature gave them a greatly improved overview of the data, especially in potentially ambiguous situations. Overall, a significant improvement in tracing performance was observed when compared to manual tracing in VR and traditional methods on desktop. One expert tester reported that "[...] the 3D navigation and interaction capabilities used in VR made the tracing experience easier and more intuitive, compared to the large number of view point adjustments that must be made when tracing on a desktop."

Virtual Reality Exposure Therapy for Social Anxiety Disorder: A Systematic Review and Meta-Analysis

VR has also found applications in the field of psychology. One such use case is Virtual Reality Exposure Therapy (VRET) in which patients are subjected to environments which can trigger their disorders, allowing them to interact with them in a safe manner. Horigome et al. [6] conducted a systematic review and meta-analysis on the use of VRET for Social Anxiety Disorder (SAD) in 2020. Their analysis shows that VRET has similar efficacy to In Vivo Exposure Therapy (IVET) while also retaining similar dropout-rates. The long-term efficacy of VRET might be lower compared to in vivo experiences, but still showed significant improvements over the control groups. In this case the control group consisted of patients on the waiting list or those continuing their existing treatment as usual. The observed long-term efficacy of VRET is combined

with the benefit of potentially much-reduced overhead in setting up realistic-enough scenarios to "provoke the appropriate level of fear response in patients"; in the specific case of [SAD](#), no (or much fewer) real people need to be gathered in order to recreate social contexts such as public speaking in [VR](#). The authors also note that the quality of the [VR](#) hardware might have an impact on the quality of the treatment due to the "feeling of presence" being lower with a lack of immersion ("realness subscale"), however further research is needed to answer this question.

2.4 Virtual Reality Design Challenges

[VR](#) technology shows huge potential for providing new possibilities in a variety of domains. However, it also comes with a number of challenges, some of which are unique to the immersive environments or the required hardware.

Creating Augmented and Virtual Reality Applications: Current Practices, Challenges, and Opportunities

In order to get a sense of the challenges faced by [VR/AR](#) developers, Ashtari et al. [2] conducted a semi-structured interview study on the topic in 2020. The authors identified eight key issues and challenges that most of the 21 developers interviewed agreed on. The subjects were from a range of backgrounds and can be separated into three groups with roughly equal splits: Professional designers, hobbyists, and domain experts.

To start, the participants expressed difficulty with knowing where to start. Since the technology for [VR/AR](#) is still evolving, knowing the state-of-the-art can be difficult, and understanding the nuances of the hardware "was particularly problematic". A lack of experience in developing software in this area further adds to the confusion at the start. This ties into the next point highlighted by the participants: A lack of useful online learning resources and difficulty using the ones available. [VR/AR](#) software is still a relatively niche area of development, and since the hardware is not necessarily interoperable by default, resources that are applicable to one platform can be useless for another. Furthermore, even newer hardware or software revisions of the same platform can lead to outdated and misleading information being available. Another area for confusion is the lack of design guidelines and established practices. The few that do exist are "not only scattered all over the Web but also not suitable for supporting many complex scenarios." This is further compounded by the fact that different hardware platforms come with potentially different input methods, further complicating portability. One point that is mostly unique to [VR/AR](#) is the physical aspect which has to be considered in the design. As noted by the authors "participants described how user research for [VR/AR](#) involved designing the posture of users, reducing fatigue, and eradicating simulators' sickness." This aspect closely interacts with the developers having to consider the environments in which the application may be used. Anticipating and accounting for different use case scenarios is described as particularly challenging for developing [AR](#) applications. Debugging [VR/AR](#) applications is another key barrier to development. Due to the many unknowns such as the dimensions of motion possible in immersive environments, thoroughly testing an application becomes quite time consuming if not impossible. Another point of note is "the constant transition between the virtual world and the debugging console caused nausea and fatigue among [AR/VR](#) creators [...]". The authors conclude that more robust [VR/AR](#) toolchains are needed with better integration for debugging and testing.

VR Locomotion in the New Era of Virtual Reality: An Empirical Comparison of Prevalent Techniques

One example of the unique challenges faced by VR developers is the implementation of virtual movement controls. Boletsis et al. [3] compared the three most prevalent solutions in 2019, namely Walking-in-place, controller/joystick movement, and teleportation. Walking-in-place requires the user to perform physical movements that are meant to mimic walking, such as lifting ones legs and/or swinging ones arms back and forth. The speed at which these actions are performed determine the movement speed in virtual space. Controller/joystick movement relies on a directional analog input to determine the speed and direction of movement, a method that is well established for video games. And lastly, for teleportation the user points at a virtual location and their virtual viewpoint is instantaneously teleported there. This instant transition leads to noncontinuous movement, requiring the user to re-orient themselves after each transition. The other two methods in contrast provide continuous movement, visually showing the virtual movement to the user as it happens.

In order to evaluate the methods the authors conducted both questionnaires as well as semi-structured interviews with 26 participants. The test itself consisted of a virtual environment in which the subjects had to find specific locations, requiring them to navigate the entire space. Walking-in-place was generally considered the most physically demanding while also offering the highest immersion. However, it also caused some participants to suffer from motion sickness and caused fear of collision with real-life objects. Controller/joystick movement was considered very easy to use and offered satisfactory immersion. Users did however report the highest amounts of motion sickness from using this movement mode, although this phenomenon was only temporary. While teleportation was similarly easy to use as controller/joystick movement, the noncontinuous motion lead to the lowest immersion in participants. Overall, the authors conclude that each method has specific advantages and disadvantages. These may be addressed individually with specialized implementations and changes from their most basic forms as implemented in this study.

3 Background

Some of work described in this thesis requires additional technical information which is outlined in this chapter.

3.1 Celerity's Structure

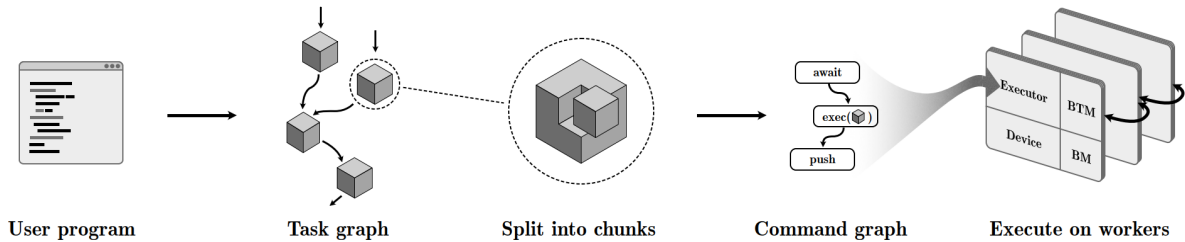


Figure 2: Outline of the Celerity runtime [10]. A user program is split into a task graph based on data dependencies, with the tasks internally subdivided further into chunks for distribution to the individual compute nodes. The command graph is generated based on these chunks and contains all the data transfer operations needed to satisfy the dependencies. This sequence of commands is then executed by the worker node.

The basis for this thesis is the Celerity project, as highlighted in section 2.1.

A Celerity execution, illustrated by figure 2, starts by constructing a *task graph* which captures the coarse dependencies between kernels based on their buffer access patterns, very similarly to how SYCL itself handles dependencies between kernels. However, thanks to the additional information provided by the user through the range mappers, it can interleave data-independent executions even if they operate on the same buffer. Based on the task graph a detailed *command graph* is generated where each command is a concrete operation, such as a computation or data transfer. This graph contains all the actions the individual compute nodes have to perform in order to complete the distributed computation while keeping the required data up-to-date and consistent across all nodes. Each command contains enough information for the worker nodes to determine when it is ready to execute and which parts of data it depends on. Once all the dependencies are fulfilled the work is executed as soon as possible with the individual workers being responsible for scheduling the work to the compute devices available to them. When a command is executed it gets converted into a *job*, essentially a wrapper that allows for quick polling of its status.

All of this information is stored in the execution trace file, whose structure is described in detail in section 4.2.1. The internal workflow of Celerity aids in understanding why for example the timing information of the different commands is contained in job objects, and not in combination with the commands themselves.

3.2 Virtual Reality Technology

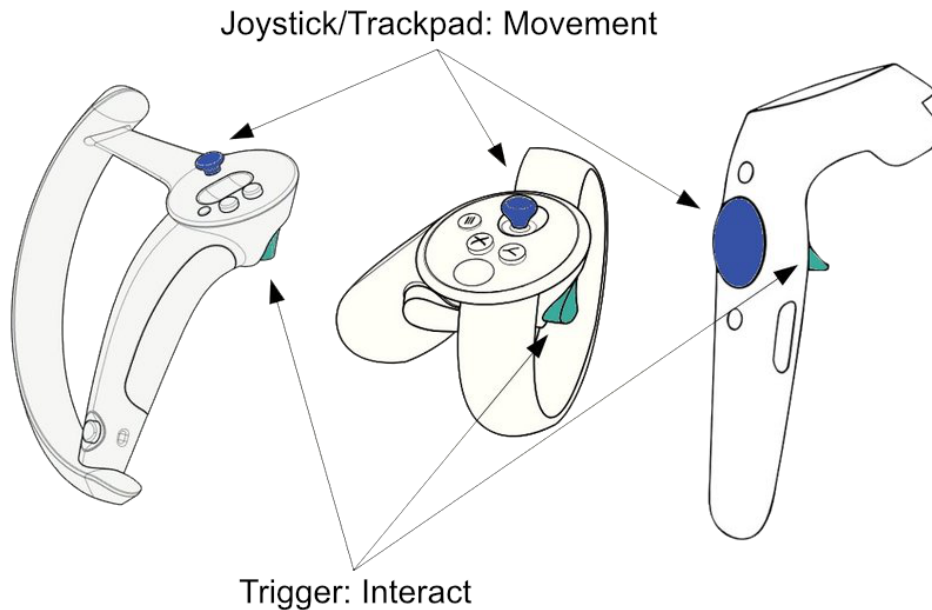


Figure 3: Comparison of three different VR controllers (adapted from [4]) with the controls for the two input modes used by the VR trace visualizer, movement and interaction, marked on each. From left to right: Valve Index, Oculus Touch (similar to Meta Quest 3), and HTC Vive. While visually different they all share a trigger for the index finger, and a joystick (or touchpad) for the thumb.

Virtual Reality aims to simulate a virtual environment and allows the user to interact with it in real time, creating an immersive experience. This results in specific hardware and software requirements:

1. An HMD to display separate images to each eye of the user to create a stereoscopic picture
2. Some form of input devices to allow the user to interact with the scene
3. A way to track the position of the HMD and input devices in physical 3D space
4. A compute device capable of running software integrating the tracking data with a simulated environment while maintaining relatively high frame rates (>60 frames per second (FPS))

While the 1st and 3rd requirements are solved by the hardware manufacturers, the other two are of significance for the application developed in this thesis. Especially the software requirement for high performance has to be considered in a number of design decisions throughout the project.

The first VR devices were built in the 1970s for the US military, mostly for use in flight simulators¹. Since then numerous different solutions to the previously mentioned requirements were tried and tested. In the last decade VR technology became readily available on the consumer market, with most offerings sticking to a specific form factor: A relatively light-weight HMD combined with specialized controllers for both hands, consisting of buttons and joysticks commonly found in video game console controllers. Some examples

¹<https://web.archive.org/web/20150821054144/http://archive.ncsa.illinois.edu/Cyberia/VETopLevels/VR.History.html>, accessed 28.01.2025

of modern VR devices are the HTC Vive², the Valve Index³, and the Meta Quest 3⁴. Figure 3 shows the (left-hand) controllers of all three of these with two main features visible: A trigger for the index finger, and a joystick (or touch pad in the case of the HTC Vive) accessible by the thumb. These controllers are mirrored for the other hand, giving two sets of these controls. Since these features are present on the vast majority of modern VR controllers, relying only on these gives the trace visualizer inherent portability across hardware platforms.

Virtual Reality technology comes with some other unique properties which need to be considered when building an application: For one, the user is able to move in physical space, giving them an essentially unlimited range of motion (only limited by the room around them, which the software cannot control). This concept is called the *play-space*. While some of the more extreme situations (e.g. the user lying on the floor) do not have to be accounted for in this case, the application should function with a range of possible user "arrangements" - most importantly both standing and sitting poses. Another property is the limited tracking accuracy of the controllers: Due to imperfections in the systems used to locate the HMD and controllers in physical space artifacts such as controller "jitter" (i.e. small movements of the controllers without physical movement) and tracking loss are possible. While full tracking loss is essentially impossible to compensate for, controller "jitter" can be mitigated using a so-called "deadzone" during relatively fine-scale operations, as is detailed in section 4.4.1.

3.3 Unity Engine

The Unity Game Engine⁵ is a free-to-use (for hobby and scientific works) development tool for creating all kinds of real-time applications. It offers tools for both 2D and 3D games, as well as native support for AR, Mixed Reality (MR), and VR. Given its long-standing reputation as an easy-to-learn game engine, it retains a large online community providing support, as well as an excellent official documentation in the form of the Unity Manual⁶. It also ships with an extensive built-in profiler for analyzing the runtime performance of the application. All of these factors made it an easy choice to use for this project.

The most important concept throughout the engine is the object-component system. Objects, or more specifically *GameObjects*, encapsulate all aspects of a scene in Unity. Each object consists of a multitude of *components* making up all of its properties and behaviors. For example, a simple cube would have a transform component describing its position and orientation in space, a mesh component containing the mesh data of the cube, and a mesh renderer which forwards that data to the rendering pipeline (heavily simplified). This object-component system is combined with a *scene hierarchy*, allowing game objects to be parented to other objects in the scene. These parent-child relations also allow objects to inherit some of their parent's properties, such as their position and rotation in space.

While Unity is using the C++ programming language internally, it offers a scripting interface utilizing C# for programming the game components. As such, all code for the application as well the ones shown in this thesis are in C#.

²<https://www.vive.com/de/>, accessed 28.01.2025

³<https://www.valvesoftware.com/de/index/headset>, accessed 28.01.2025

⁴<https://www.meta.com/at/quest/quest-3/>, accessed 28.01.2025

⁵<https://unity.com/products/unity-engine>, accessed 29.01.2025

⁶<https://docs.unity3d.com/Manual/index.html>, accessed 29.01.2025

3.4 OpenXR and Unity

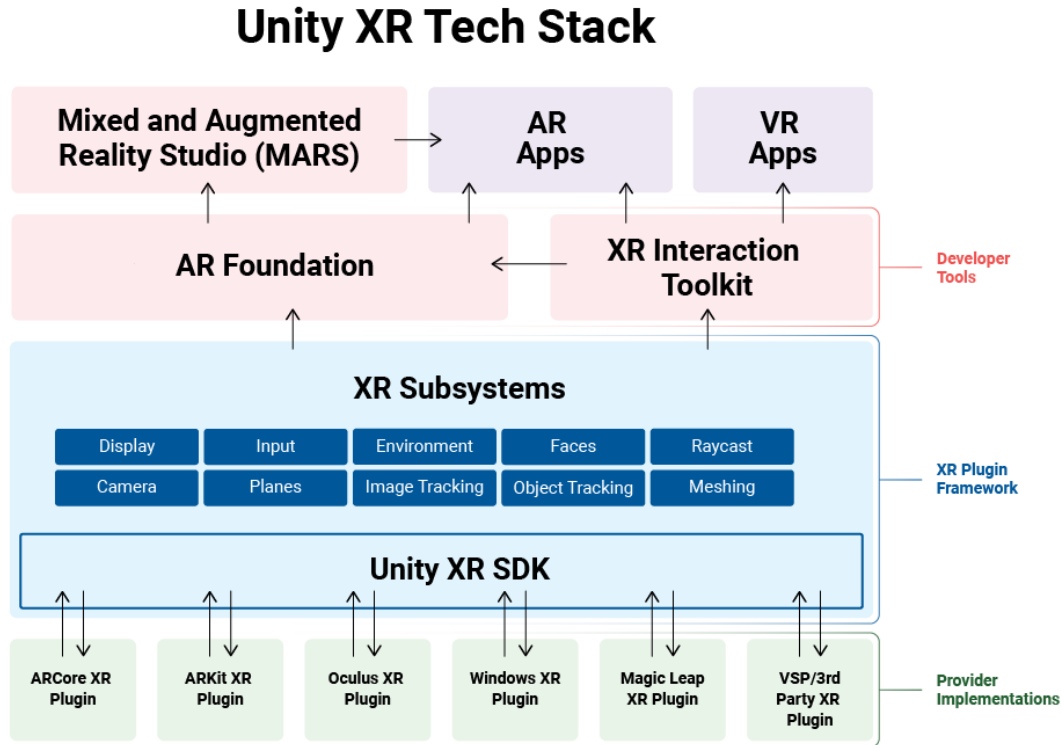


Figure 4: Architecture overview of the Unity XR Technology Stack. Different XR provider implementations interact with the Unity XR SDK and are presented to the developer transparently. For this application only the VR-relevant components are used. Image taken from the Unity Manual website on the topic⁷.

As described in section 2.4, XR (umbrella term for VR, MR, and AR), and more specifically VR applications, come with unique design challenges. One potentially daunting aspect of XR development is the large number of hardware and software solutions available on the market. Choosing one or a few of them to support is challenging since the market continues to evolve. Thankfully, the OpenXR standard⁸ developed by the Khronos Group provides a unified interface for most VR solutions on the market, effectively eliminating the need to support each hardware solution individually.

The Unity engine provides their so-called Unity XR Tech Stack⁹, abstracting away the different VR hardware and software ecosystems on the market through the use of XR plug-ins. An overview of this architecture is shown in figure 4. Although most hardware is OpenXR compatible, some manufacturers still provide their own plug-in implementations, which might offer more functionality in some cases.

⁷<https://docs.unity3d.com/6000.0/Documentation/Manual/XRPluginArchitecture.html>, accessed 01.03.2025

⁸<https://www.khronos.org/openxr/>, accessed 29.01.2025

⁹See footnote 7.

For the development of this project a Valve Index¹⁰ VR headset is used. The OpenXR plug-in¹¹ in combination with Unity is used to map to the SteamVR¹² runtime, which in turn interacts with the headset. This setup worked out-of-the-box with no issues during the development of this project. The provided XR interaction toolkit¹³ comes with the basic elements for interacting with the scene, most notably the player movement and interactions with user interface (UI) panels. It also provides a uniform interface for controller inputs, mapping the controls described in section 3.2 to common endpoints in the Unity input system.

¹⁰<https://www.valvesoftware.com/de/index/headset>, accessed 28.01.2025

¹¹<https://docs.unity3d.com/Packages/com.unity.xr.openxr@1.13/manual/index.html>, accessed 29.01.2025

¹²<https://www.steamvr.com/en/>, accessed 29.01.2025

¹³<https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.1/manual/index.html>, accessed 01.03.2025

4 Implementation

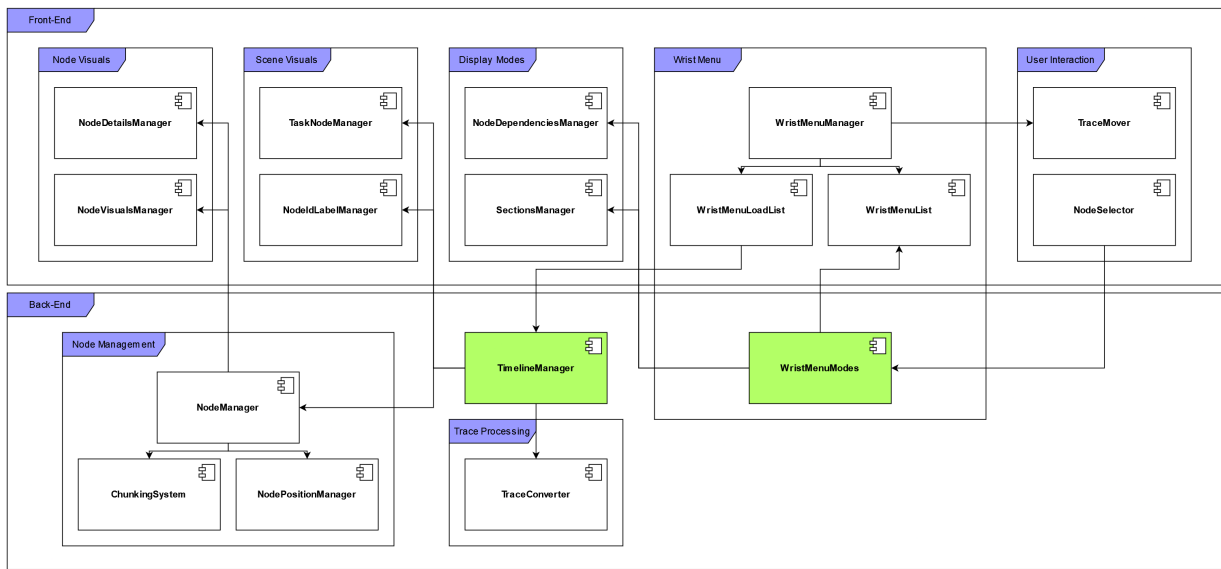


Figure 5: Components diagram of the architecture. Note the two layer design, as well as the two main coordination points marked in green.

There are three main aspects to consider during the implementation of the project: The software architecture, user experience, and performance.

The project is built as a two-layer architecture, as shown in figure 5. Different components are designed to be as independent from each other as possible in order to allow for relatively easy modification without impacting the rest of the application, in line with the classic SOLID principles. At the same time, the user experience should be as intuitive and seamless as possible. Furthermore, the visualizations should be clear and the tools easy to use. While these metrics are hard to measure without an in-depth user study, they are still continuously evaluated during development by frequent testing. And finally, the goal is to avoid reducing "the interactivity of a VR application to a slow slide show", as put by Su et al. [12]. While performance is not the main goal of this project, since it is mostly a proof-of-concept, it is still important in order to enable a good user experience. In practice the overall performance is managed by making informed decisions when choosing data structures (see section 4.2.3) and optimizing the visualization components where needed (see section 4.3.1 for an example).

4.1 Overview

The project consists of several distinct components which are shown in figure 5. Together they handle the following tasks:

Trace data pre-processing The raw trace data is processed, implicit data reconstructed, checked for integrity, and stored in an internal data structure. This is handled by the Trace Processing component and only done when a new trace is loaded.

Visualizing the structure Two main aspects make up the structure of a Celerity trace: The task and command graphs. The task graph acts as a higher-level view of the execution and in the context of this project serves as a general reference to the user. The command graph contains the main information about the execution and is visualized in full. This is handled by two distinct components, which are further detailed in section 4.3.1: The Node Management, and the Node Visuals.

Visualizing the details Some of the more in-depth information (e.g. inter-command dependencies, buffer accesses, idle sections) is only shown when specific display modes are active in order to prevent visual clutter. These visualizations are handled by the Display Modes component, and where applicable, the Node Visuals component.

User Interface The user interacts with the program in two main ways: Using the wrist menu (which is, as the name implies, attached to the left wrist of the user), and by directly interacting with the 3D scene. These two methods are handled by the Wrist Menu and User Interaction components respectively.

These overarching tasks are coordinated by two main components: The Timeline Manager, responsible for general scene management, and the Wrist Menu Modes, which handles all of the visualization modes. These two are highlighted in green in figure 5.

At runtime the program follows a set order of operations:

1. The user selects a trace file to load and which visualization style to use.
2. The trace file is loaded and the pre-processing is performed.
3. The trace structure visualization is generated.
4. The user is free to manipulate the scene and switch between visualization modes.

4.2 Trace Data

The foundation of the project is the trace data recorded during a Celerity execution. In this section the parsing, pre-processing, and internal storage of this data is described.

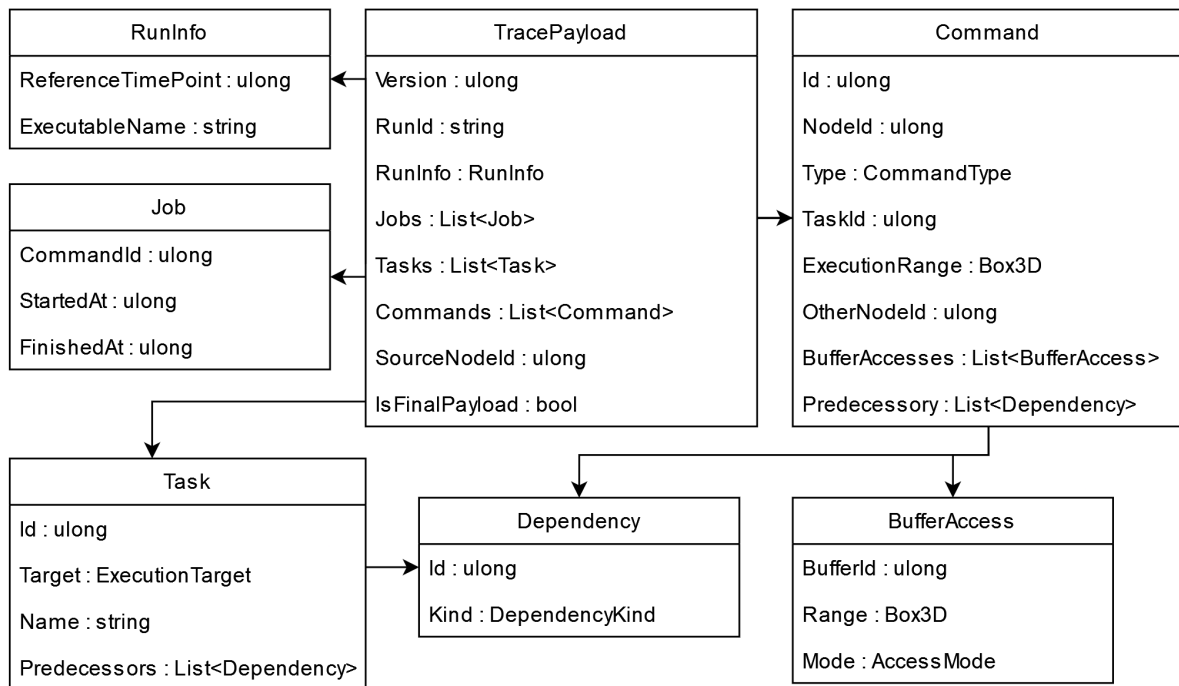


Figure 6: UML class diagram of the raw trace data. The trace file itself consists of multiple TracePayloads. Some parts have been omitted for clarity.

4.2.1 Trace Data Format

The trace data is recorded by the Celerity system at runtime and stored as a ProtoBuf¹ file, the structure of which is shown in figure 6 (with some details omitted for clarity). The trace file contains all of the information needed to reconstruct the runtime behavior of the system. Internally the trace consists of multiple TracePayloads, one for each compute node that participated in the execution.

The data is parsed using the C# ProtoBuf library *protobuf-net*² in combination with a schema file defining the data layout. The parser maps the file contents to C# classes matching the schema of the data, which have to be provided separately. Luckily, there are automated tools³ available online for generating these. Once converted the data is processed into other formats as required by the project at runtime.

There are some things of note in the structure of the raw trace data:

- Each compute node (denoted by the SourceNodeId) creates it's own TracePayload, meaning that the commands are scattered across the payloads
- Timing information is stored in the form of Jobs and is always relative to the ReferenceTimePoint of that compute node, as well as separately from the commands themselves

¹<https://protobuf.dev/>, accessed 17.02.2025

²<https://github.com/protobuf-net/protobuf-net>, accessed 17.02.2025

³<https://protogen.marcgravell.com/>, accessed 17.02.2025

4 Implementation

- All dependency information is saved as predecessor relations, i.e. back-to-front from an overall perspective
- Tasks do not have a recorded duration; instead, their extent is defined by the commands within the task
- Buffers are only stored implicitly through BufferAccesses; their full dimensions have to be reconstructed from all the accesses combined

These properties closely reflect the inner working of the Celerity system at runtime as described in section 3.1. However, they also make using the raw trace data impractical for the trace visualization system.

4.2.2 Pre-Processing

In order for the main visualization system to work efficiently, the raw trace data needs to be processed and converted into an internal data representation more suited to the access patterns done by the system. In the end none of the original data structures and classes are used. By only relying on internal data structures the system is decoupled from the format in which the traces are stored. If more information is provided in the future or a different data format is chosen, the system's internal implementation can remain the same. The only modification necessary to incorporate a new format would be a new implementation of the TraceConverter component.

Parsing and pre-processing is done in four steps:

Step 1 - Structure

As a first step the tasks and commands are extracted from the traces, together with auxiliary information about the trace such as the name of the executable and execution arguments.

The tasks and commands are scattered across the different TracePayloads. They are each collected and checked for duplicates based on their unique ids. At the same time information about the runtime system is gathered. The number of compute nodes and their ids are implicitly stored as part of all the commands, so they are collected and explicitly stored as ComputeNode objects. The same is true for Buffers, which need to be reconstructed from all BufferAccesses in the original trace. Buffer objects are created based on the buffer ids, and their minimum and maximum extends are computed from the minimum and maximum access ranges from all BufferAccesses referencing those ids. These ranges are simultaneously converted into more Unity-friendly representations using *Vector3Ints* and the resulting BufferAccess objects are stored with their corresponding commands. In order to increase the robustness of the system, any invalid references to non-critical parts of the trace (e.g. to non-existent task ids) are logged and subsequently ignored. This allows the system to function with traces that only consist of partial information without too much loss of functionality.

Step 2 - Dependencies

With the commands extracted and their information converted into internal data structures, the inter-command dependencies are extracted. The trace files contains this information in the form of predecessor-dependencies. However, since the visualization system frequently requires data on both predecessors and successors, the internal data structure keeps track of both. This effectively turns the command data structure into a bi-directional graph structure with each one pointing to a number of predecessor and successor commands. It should be noted that these are not direct references or pointers, but instead *Dependency* class types, since

dependencies can have different kinds of relationships. Each Dependency models a predecessor-relation consisting of an origin and a target, with the former being *later* in the timeline and the latter being *earlier*. Therefore the command itself is always the origin for all predecessor dependencies it is keeping track of, and the target for all successor ones. Tasks also contain dependency information, which is handled in the exact same way as the command dependencies with the dependency class accepting a template parameter.

Step 3 - Execution Times

Next up the command execution times from the trace are extracted and normalized. Each compute node creates its own TracePayload during runtime and records the commands it executed together with their start and stop execution time stamps, stored in Jobs data structures. The recorded time stamps are relative to the node's execution start time, called the *reference time stamp*. However, since the compute nodes don't necessarily start at the exact same time, their execution start times will be different. For simplicity the starting point of the first compute node (i.e. with id 0) is chosen as a global starting time reference. All other execution time stamps are then adjusted to reference this global starting time. It should be noted that this approach assumes that the system clocks of all compute nodes are synchronized in order for the times to be comparable. Once a common reference point is established the Jobs from all TracePayloads are extracted and associated to their corresponding commands. The start and end time stamps are adjusted to the common reference and stored in the command data structure. At the same time the overall starting and ending times of the trace are collected and stored for later use.

Step 4 - Data Dependencies

The data dependencies between commands are only implicitly contained in the trace data, similarly to the Buffers and their sizes. They are reconstructed from the relationships between Push and AwaitPush commands. Each Push command has an associated BufferAccess with Read access mode, as well as a target compute node id to which the data is sent. The corresponding AwaitPush command has an identical BufferAccess, except that the access mode is DiscardWrite, as well as a origin compute node id from which to receive data. However, these relations are not necessarily unique, since the same data transfer can occur multiple times during the recorded execution. In order to match them uniquely, the temporal relations of Push and AwaitPush commands are considered. An AwaitPush command starts as soon as the compute node is ready to receive the data. The corresponding Push command starts when the data to be sent is available on the sending compute node. This results in potentially large differences in StartTimes between the two commands. However, the EndTimes tend to be closer together since both of them end when the data transfer is completed. It should be noted that in practice, the Push command ends before the corresponding AwaitPush, but this is not guaranteed to happen. Therefore, the combination of Push/AwaitPush commands with the minimum absolute difference in EndTime is chosen as the unique matching pair. Based on this match the data dependency is added to both commands as a predecessor and successor dependency, respectively. It should be noted that this method of determining matching transfer commands is not guaranteed to accurately reconstruct the data dependencies: It is possible to conceive a scenario where multiple data transfers of the same buffer chunk between the same compute nodes happens in rapid succession, resulting in a potential mismatch between commands. However, this possibility seems sufficiently unlikely in practice to cause concern. Furthermore, the current trace data format does not contain enough information about data transfers to allow for a better method of reconstruction.

4 Implementation

4.2.3 Internal Data Structure Design

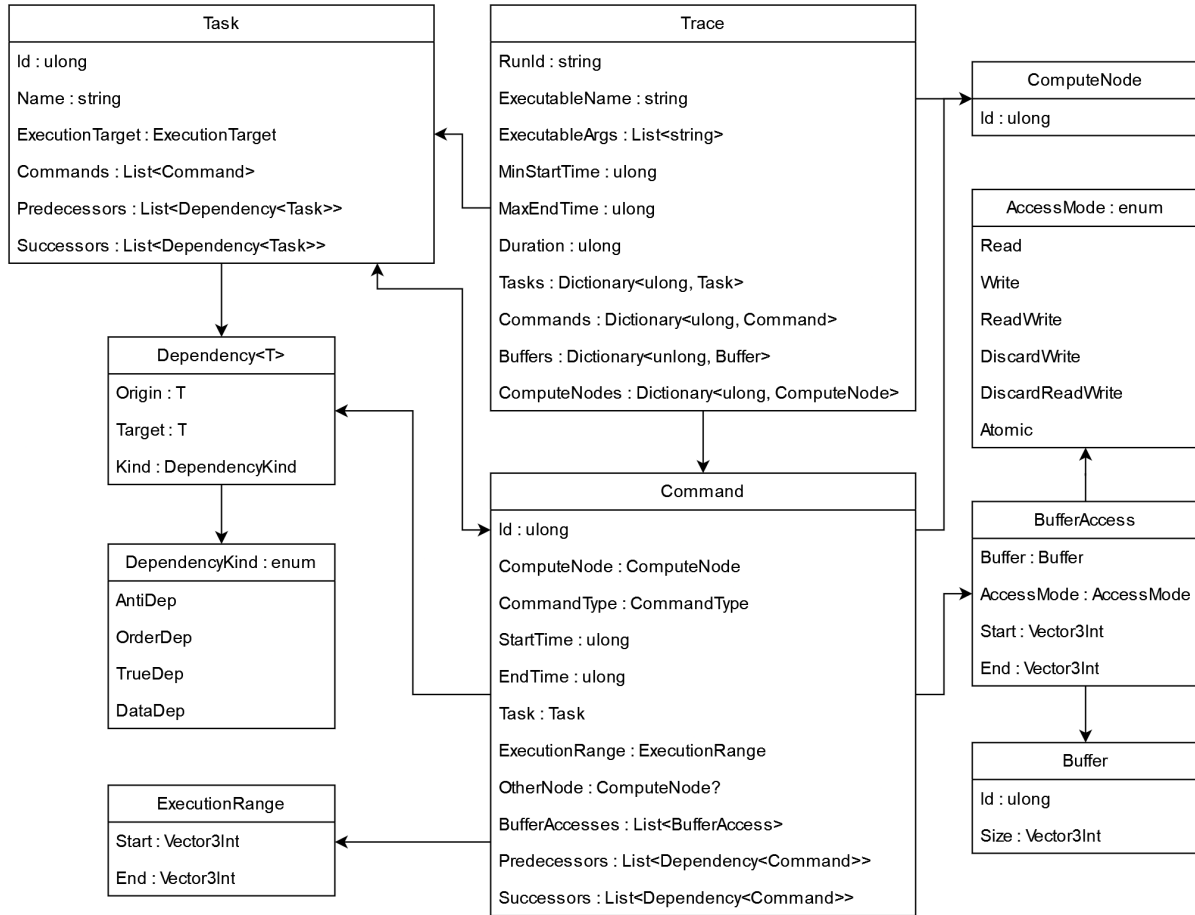


Figure 7: UML class diagram of the Trace data structure. While it is similar to the structure of the raw trace file, it allows for additional inferred data to be stored. It also provides additional cross-references to allow fast traversal of the data for common access patterns (e.g. both predecessor and successor lists in the Command class).

The heart of the project is the Trace data structure, which is shown as a class diagram in figure 7. It acts as a container for all of the extracted and computed trace data from the previous section. As apparent from its structural overview in figure 7, the internal design is very similar to the raw trace data. However, as mentioned before, the explicit decoupling of the internal data structure from the trace file format allows for relatively easy integration of future format changes.

Additionally, this separate data structure allows for the inferred and additionally computed information to be stored in the same place, as well as in a format conducive to the access patterns present in the application. As an example, the original trace data only contains references predecessors for the commands, while the internal command class provides both predecessors and successors. These two lists together allow for a "lateral" traversal of the commands dictionary along the dependency relations, which is necessary for the visualization of recursive command dependencies as described in section 4.3.3.

Another optimization is the design of the CommandType enum. It acts as a bit field, as shown in figure 8,

CommandType : enum	
None	= 00000000
Nop	= 00000001
Horizon	= 00000010
Task	= 00000100
Push	= 00001000
AwaitPush	= 00010000
Shutdown	= 00100000
Sync	= 01000000
All	= 01111111

Figure 8: Detailed view of the CommandType enum, allowing it to act as a bit field.

allowing it to be used for the creation of bit masks using binary operations. This feature is heavily used in the command type-dependent filtering visualization modes. An example of its use is shown in listing 1.

One thing of note in the Trace data structure is the use of Dictionaries for tasks, commands, Buffers, and ComputeNodes. All of these map from the unique ids of each type to their corresponding value. A more space-efficient way to store these would be to use simple arrays where an element's index corresponds to their id. While Celerity generates all ids in a deterministic manner, the trace files might be incomplete, and therefore some parts of the id range might be missing. Therefore, Dictionaries are used in order to guarantee generality for all id configurations.

There is one more internal data structure aside from the Trace class worth nothing: A Dictionary mapping from commands to nodes contained in the NodeManager class. This is part of the command node visualization system and is essential to quickly finding the nodes corresponding to a specific command. While each command node also contains a reference to its respective command, this dictionary enables efficient access the other way around.

As mentioned before, the choice of the specific primitive data structures for each of the components is based on the most common access patterns. For example, while the TraceConverter needs to iterate over the elements of the commands Dictionary multiple times, which is an inherently inefficient operation, it only does so during the loading of a new trace file. During runtime, when application performance is much more important due to real-time user interactions, all accesses are random access by keys, which the data structures are designed for.

4.3 Visualization

The visualization of the trace data can be grouped into three main categories:

Commands Timeline The commands are shown as colored, semi-transparent boxes (referred to as *command nodes*) which are arranged along one of the spatial axis. This is very similar in concept to the general layout of traditional 2D trace viewers.

Command Details More in-depth information about the individual commands is shown on demand. The user can select a command node in space and is shown its ID, duration, and buffer accesses. If a different visualization mode (e.g. recursive dependencies mode) is selected, some of the shown data is adjusted to match the mode. For example, when the recursive dependencies mode is active, the list of buffer accesses is replaced with a list of predecessor and successor dependencies.

Mode-Specific Information And lastly, specific information is shown in various ways depending on the selected visualization mode. These include things such as dependencies between nodes, idle sections, or selectively hiding commands by their types.

Each of these categories is highlighted in more detail in the following sections.

The scene is laid out in Unity as follows: The entire timeline with all visualization parts has one common parent object, the TraceParent. In turn, each distinct part of the visualization has their own parent object for easy separation. And finally, the visual components for the command details are parented to the respective command node object.

This scene structure allows for one of the main features of the project: Having two distinct layout variants. When loading a trace file, the user can select to use either a linear layout (analogous to traditional 2D trace viewers), or a novel circular timeline layout in which the command nodes are arranged around the user with the timeline extending upwards (see figure 10).

Another important concept is the timeline origin: It represents the center of the scene and is shown as a gray circle on the ground. At the start of the program the user's playspace (see section 3.2 for more information) is centered on the origin, and when a timeline is loaded it is set to start there as well. When using the circular layout variant, the origin is also offset slightly vertically (with the gray indicator disk remaining at ground level). This allows the timeline to start at about chest-level for the user, which brings it more in line with their actual viewpoint. It should be noted that the system is built to allow this vertical offset to be reconfigured, although this setting is not exposed to the user at runtime.

4.3.1 Commands

Commands are visualized by their command nodes: Semi-transparent square rectangular cuboids, or put simply, boxes. These boxes are colored depending on the command's type, as shown in figure 9. Their "length" along the primary axis is adjusted to match the execution time of the command at runtime, at least as closely as possible. There are some usability concerns that have to be addressed in the scaling and positioning of the nodes: In order to differentiate two adjacent nodes a small gap is inserted between them, and nodes of a very small length would visually disappear, so a minimum length is enforced. In the case of multiple subsequent small-length nodes, this minimum length forces them to be distributed over multiple vertical layers to separate them (see section 4.3.1 for more information on node stacking and layers). Additionally, each command node also contains a Unity box collider, essentially providing the built-in physics engine with its dimensions. This is essential for the node selection process described in section 4.4.1 to work.

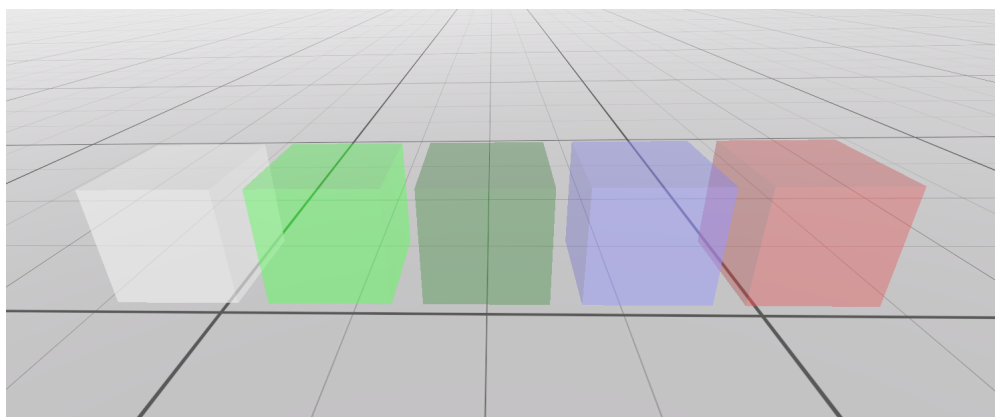


Figure 9: Command nodes are colored depending on their type. From left to right: Task, Push, AwaitPush, Horizon, and Sync (some types are omitted for clarity).

Command Node Rendering Optimization

While coloring a box is relatively simple to do in the Unity engine, doing so efficiently on a larger scale can present a challenge. For context, Unity renders objects in the scene using draw calls to an underlying graphics API⁴. These draw calls include all the information needed to render an object on the screen, making up the render state. Changes to this state tend to be resource intensive, and especially in a case like this where the goal is to render a lot of very simple boxes on screen, they can take up a significant amount of time. Testing early on in the development of the project showed that by simply changing the color of a box the naive way (by changing the box's material at runtime), Unity generates a unique render state *for each box that needs to be rendered*. Unsurprisingly, this led to a significant performance issue, though one easily diagnosed with the excellent built-in profiler. The solution is to manually prepare unique materials for each of the colors (as well as another copy for a non-transparent version used for node selection in section 4.3.2). These pre-made materials are assigned to the boxes at runtime to change their colors, allowing Unity to recognize that they can be rendered with the same render state. This optimization in theory cuts down the number of draw calls from one for each command node to two for each command type. However, in practice the Unity engine still issues more draw calls than logically necessary, though by far not as many as one per node (see section 5.3.3 for more details).

Chunking System

Since there are potentially quite a lot of commands in a trace file (multiple 10,000s), the rendering optimization mentioned before does not suffice to ensure good run-time performance in practice. Therefore a separate system for only rendering part of the timeline is implemented, called the Chunking system. During the command node creation and placement, the nodes are sorted into equally-spaced chunks based on their center point (halfway between start and end of the execution duration). Each chunk keeps track of the nodes assigned to it, as well as all other chunks whose nodes either partially or fully overlap it as a set. This is done by going through all chunks that lie within the range of the time from command StartTime to EndTime and assigning the current chunk to those as an overlapping one. Keeping track of these overlaps is necessary since nodes can have a duration that is larger than the overall size of the chunk they are assigned to (i.e. very long-running

⁴<https://docs.unity3d.com/Manual/optimizing-draw-calls.html>, accessed 04.11.2024

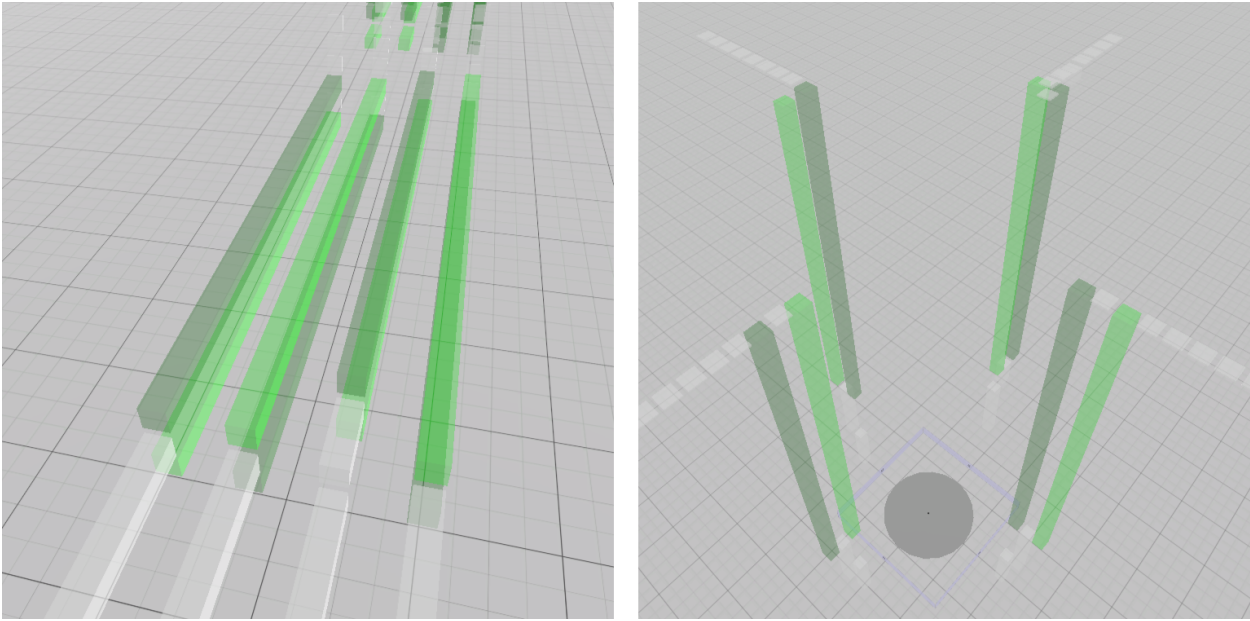


Figure 10: The two timeline layout variants. On the left: Linear layout with command nodes extending along the primary axis and compute lanes along the depth. On the right: Circular layout with command nodes extending upwards and compute lanes arranged in a circle around the origin.

commands). When a certain range of the timeline is shown, all of the nodes in that time frame should be shown, including the ones that: 1) Start long before and end within the time frame; 2) Start within the time frame but run for a while afterwards; 3) Start before and end after the time frame, with their center point lying outside the shown range; and 4) Start and end within the time frame. By keeping track of chunks whose nodes overlap a given chunk, all of these cases are essentially pre-computed. When a specific range of the timeline should be shown at runtime, the chunks that cover that range are shown, as well as all the chunks that they have listed as overlapping them. In practice this ensures that even long-running command nodes are shown, at the cost of chunks outside of the immediate time frame being visible as well. The feature of only showing part of the timeline can be turned on and off by the user with the so-called range filter, which is discussed in section 4.3.5.

Command Node Positioning and Layouts

Commands executed by different compute nodes are placed in separate locations, referred to as "compute lanes" or simply "lanes". For the linear layout the lanes are spaced along the secondary horizontal direction, whereas for the circular layout they are arranged in a circle around the origin with equal angles between them. This difference is shown in figure 10.

It is possible (and almost always the case) that multiple commands are executed on the same compute node at the same time. In this case the command nodes are stacked on top of each other in so-called *layers* (upwards for the linear layout, outwards for the circular). These layers are computed by going through the existing ones from bottom to top, and for each one finding the spot along its length where the new node would need to be inserted. If the previous node in that layer ended before the start of the new one, and there is enough spacing between them, and the same is true for the next node in that layer, then the placement is valid. However, if the node does not fit in that layer, the next layer up is checked in the same way. This is repeated until a layer

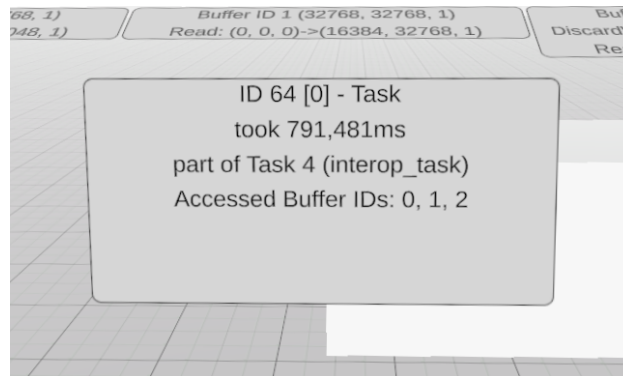


Figure 11: The command details panel. It is shown when the user selects a command node in the scene and lists general information such as the command id, duration, and type.

with enough space is found, or if none exists a new layer is added on top. In its current form the placement algorithm is optimized for placing nodes in order from first to last in the timeline. But, in the interest of making the system easily expandable, it is capable of handling nodes in an arbitrary order. It should also be noted that the resulting node placements onto layers is deterministic, but dependent on the order in which the nodes are placed, which in turn is set by the TimelineManager. In practice, this means that the same trace file always results in the same timeline layout.

4.3.2 Command Details

When the user selects a command node in space it is highlighted (shown as opaque instead of transparent) and additional command details are shown (see section 4.4.1 for more information on the selection system). These details consist of three parts: The information panel, buffer access visualization, and list menu. The first two are shown in space alongside the command node while the third is part of the wrist menu.

The information panel presents a general overview of the selected command, as shown in figure 11. It shows the command's id, duration, id of the compute node that executed it, and the Task it is a part of. If the selected command is a Push or AwaitPush type, the id of the destination or source command is shown as well, which is retrieved from the data dependency associated with the command. If a data-accessing command is selected (Task, Push, AwaitPush), a list of the ids of all accessed buffers is shown as well.

Additionally, data-accessing commands have a visualization of each accessed buffer shown. It consists of a 3D representation of the buffer as a semi-transparent cube, in which the accessed ranges are shown as sub-regions, as shown in figure 12. The transparency allows the user to see the accessed ranges within its volume, and discern potentially overlapping access ranges. In combination with the 3D cube a panel listing the accesses to the specific buffer is shown in front. This panel allows the user to quickly correlate the visual representation of the accesses with the numerical range values.

Since buffers can come in any shape and in three different dimensionalities, the system has to ensure that all of these are legible when visualized. To achieve this the cubes sides are scaled based on the relative sizes of the buffers dimensions to its largest dimension, scaled down to a fixed overall size for the cube. Additionally, an overall minimum size for the side length of the cube is enforced to prevent it from having (close to) zero-length sides. In the case of one-dimensional buffers this ensures that the "line" is visible, and in the case

4 Implementation

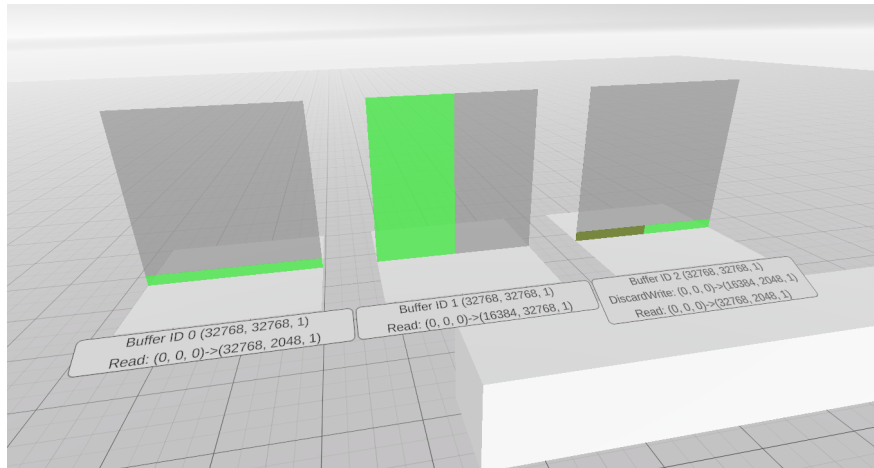


Figure 12: Buffer access visualization. Each accessed buffer is shown by a scaled cube with the accesses to it represented by colored sub-areas within it. The colors match the types of access for each buffer access.

of two-dimensional ones it provides some thickness. The result is a mostly proportionately-sized box showing the relative dimensions of the buffer.

This box provides the basis for visualizing the buffer accesses. Each access represents a sub-region of the overall buffer box. Similar to the overall buffer visualization, each access is shown as a scaled cube, which is then colored based on the type of buffer access that was performed. Read accesses are green, writes are red, read writes are yellow, discard writes are dark red, and so on. Figure 12 shows an example of multiple read and one discard write access to different buffers. Since a command might have multiple accesses to the same buffer, multiple access region boxes can be placed within the same buffer box. In general the colors are chosen to be reasonably easy to distinguish from each other, even when blended together in overlapping regions (see right side of figure 12).

In combination with the in-space representations of the buffer accesses, a list of all accesses is shown in the wrist menu. Since it is attached to the users arm and provides a scroll bar, it complements the other visualization types by giving a quick overview of all accesses. The wrist menu list is highlighted in more detail in section 4.4.2.

4.3.3 Dependencies

One piece of information critical to understanding a Celerity execution trace is the various dependencies dictating the program flow. These describe for example which commands have to finish before the next can start (true dependencies), or which data transfers have to complete before computation can continue (data dependency). Since there can be multiple dependencies for each command, and an even greater number of commands in the trace, only a subset of all dependencies is shown at a time to preserve clarity. Two main ways of showing dependencies are implemented: Recursively forwards and backwards from a selected command node, and as a critical path of the entire trace.

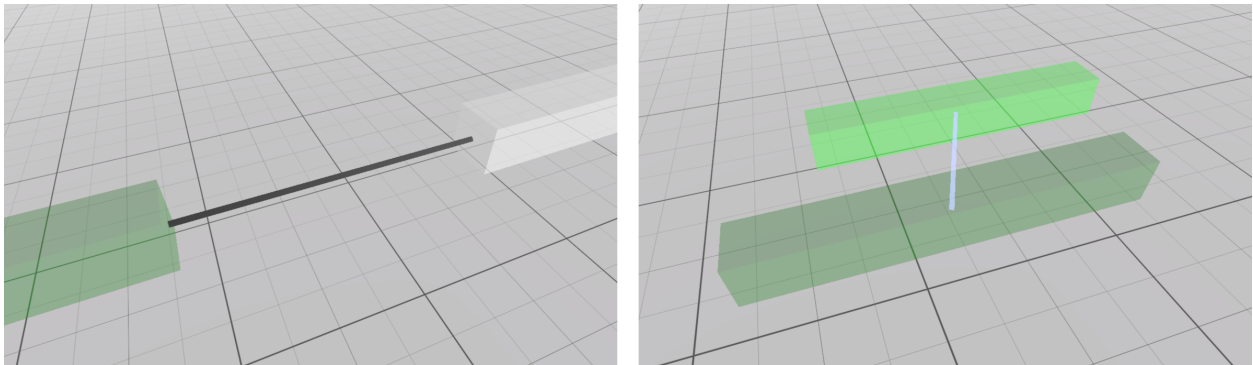


Figure 13: Dependency visualization using lines in space. On the left: A true dependency between a task and an await push command. On the right: A data dependency between a push and an await push command.

Line Drawing and Positioning

Dependencies are shown in the scene as lines between the start/end of command nodes. The Unity engine provides a line renderer⁵ component by default. It draws a thin 2D plane between the points and rotates it to continuously face towards the camera, giving the illusion of a line in space. This system is used to draw the dependency lines in the scene. In order to differentiate the dependency types visually, the lines are colored depending on their type.

The start and end points are determined based on the dimensions of the command nodes part of the dependency to be shown. As discussed in section 4.2.2, dependencies consist of an origin and a target command. However, in this context it makes more sense to think of them as the *earlier* and the *later* command in relation to the timeline. For all types except for data dependencies, the dependency line starts at the end point of the earlier command and ends at the starting point of the later command. This constellation is shown in the left half of figure 13. Data dependencies represent a special case, since it is impossible to clearly define an earlier and a later command in the relation (the target = the push-type command might start before *and* end after the await-push command since they are executed asynchronously on different compute nodes). Therefore, data dependency lines are drawn between the center points of two command nodes, as is visible on the right side of figure 13.

Since the user can show and hide a potentially very large number of dependency lines over the runtime of the program, creating and destroying the line renderer game objects every time can be resource intensive. Especially since C# is a memory-managed language with garbage collection, excess memory allocations and freeing should be avoided⁶. To that end the dependency visualization system uses object pooling. A set of line renderers is continuously reused during runtime, with more being added to the pool if the number of lines to be drawn exceeds the number of renderers available. When the dependencies should no longer be shown, the line renderers simply get disabled and left in the pool. Disabled game objects consume negligible amounts of resources in the Unity engine, with the only real impact being slightly higher RAM usage. However, the amount of memory allocations saved with this approach far outweighs the slight increase in overall memory consumption.

⁵<https://docs.unity3d.com/2021.3/Documentation/Manual/class-LineRenderer.html>, accessed 06.11.2024

⁶<https://docs.unity3d.com/6000.0/Documentation/Manual/performance-garbage-collection-best-practices.html>, accessed 08.11.2024

4 Implementation

```
1 private HashSet<Dependency<Command>> ComputeRecursiveDependencies(  
2     Command startCommand, int depth, CommandType typeMask)  
3 {  
4     HashSet<Dependency<Command>> includedDependencies = new();  
5  
6     // red/black lists for swapping  
7     List<Command> commandsA = new(8);  
8     List<Command> commandsB = new(8);  
9     // pointers to lists  
10    List<Command> previousCommands = commandsA;  
11    List<Command> nextCommands = commandsB;  
12    // initialize with starting node command  
13    previousCommands.Add(startCommand);  
14  
15    for (int d = 0; d < depth; d++) {  
16        foreach (Command command in previousCommands) {  
17            foreach (Dependency<Command> dependency in command.Predecessors) {  
18                // skip if other command is wrong type  
19                if ((dependency.Target.CommandType & typeMask) == 0)  
20                    continue;  
21  
22                includedDependencies.Add(dependency);  
23                nextCommands.Add(dependency.Target);  
24            }  
25        }  
26        // swap lists and clear next commands  
27        (previousCommands, nextCommands) = (nextCommands, previousCommands);  
28        nextCommands.Clear();  
29    }  
30  
31    return includedDependencies;  
32 }
```

Listing 1: Algorithm for recursively computing the dependencies starting at a given command, in this case for predecessor dependencies. The algorithm uses red/black swapping of the two command lists for efficiency. In each step the predecessor dependencies of all previous commands are checked and added to the set if the target command is of the correct type. For successor dependencies the predecessor and target references are swapped for successors and origin ones (omitted for clarity).

Recursive Dependency Computation

The first type of displaying dependencies is the recursive mode. When the user selects a command node with this mode active, all predecessor and successor dependencies of that command are shown. This is repeated a number of times, called the depth, for all commands on the other end of the dependencies. The recursive steps forwards and backwards are done in two separate passes in practice, although by the same algorithm. At the same time the type of all commands found to be part of the dependencies are checked against a type mask, making use of the bit field property of the command type data structure described in section 4.2.3. A simplified version of the algorithm (only for computing the predecessors) is shown in listing 1. The resulting set of dependencies is then passed to the system for drawing the lines, as well as to the wrist menu list to show them as list entries.


```

1  private List<Dependency<Command>> ComputeCriticalPath()
2  {
3      // check if cached result available
4      if (_cachedCriticalPathDependencies != null)
5          return _cachedCriticalPathDependencies;
6
7      Trace trace = TimelineManager.Instance.CurrentTrace;
8      Command endComm = trace.Commands.Values
9          .Where(x => x.EndTime == trace.MaxEndTime).FirstOrDefault()
10     ?? throw new ArgumentException("Couldn't find last command in trace.");
11
12     List<Dependency<Command>> criticalPathDependencies = new();
13
14     Command currentCommand = endComm;
15     while (currentCommand != null) {
16         // command doesn't have any predecessors -> reached the start
17         if (currentCommand.Predecessors.Count == 0)
18             break;
19
20         // exclude anti-dependencies
21         var predecessorCommands = currentCommand.Predecessors
22             .Where(x => x.Kind != DependencyKind.AntiDep).Select(x => x.Target);
23         // order by end time and select command with maximum one
24         List<Command> predCommsOrderedByEndTime = predecessorCommands
25             .OrderBy(x => x.EndTime).ToList();
26         Command nextCommand = predecessorCommands.OrderBy(x => x.EndTime).ToList().Last();
27
28         // get dependency with nextCommand as target
29         Dependency<Command> nextDependency = currentCommand.Predecessors
30             .Where(x => x.Target == nextCommand).FirstOrDefault();
31         // add dependency to critical path
32         criticalPathDependencies.Add(nextDependency);
33
34         currentCommand = nextCommand;
35     }
36
37     // cache result
38     _cachedCriticalPathDependencies = criticalPathDependencies;
39     return criticalPathDependencies;
40 }

```

Listing 2: Algorithm for computing the critical path. The result is cached for any future calls to the method and cleared when a new trace file is loaded (not shown). Some of the edge case handling is omitted for clarity.

Critical Path Computation

The second type of showing dependency data is by use of the critical path. It represents the sequence of commands within the trace that dictate the overall execution time, or in other words, the commands that the rest of the system has to wait for. Computing the critical path is relatively simple given the end times as well as the dependency relations of all commands, both of which are present in the trace data. Starting at the last command in the trace, all target commands in the predecessors are checked and the one with the maximum end time is selected as the next command in the path. This is repeated until the start of the trace is reached (i.e. until a node with no more predecessors is found). The list of dependencies traversed along the way is the resulting critical path. A simplified version of the algorithm (without some edge case handling) is shown in

4 Implementation

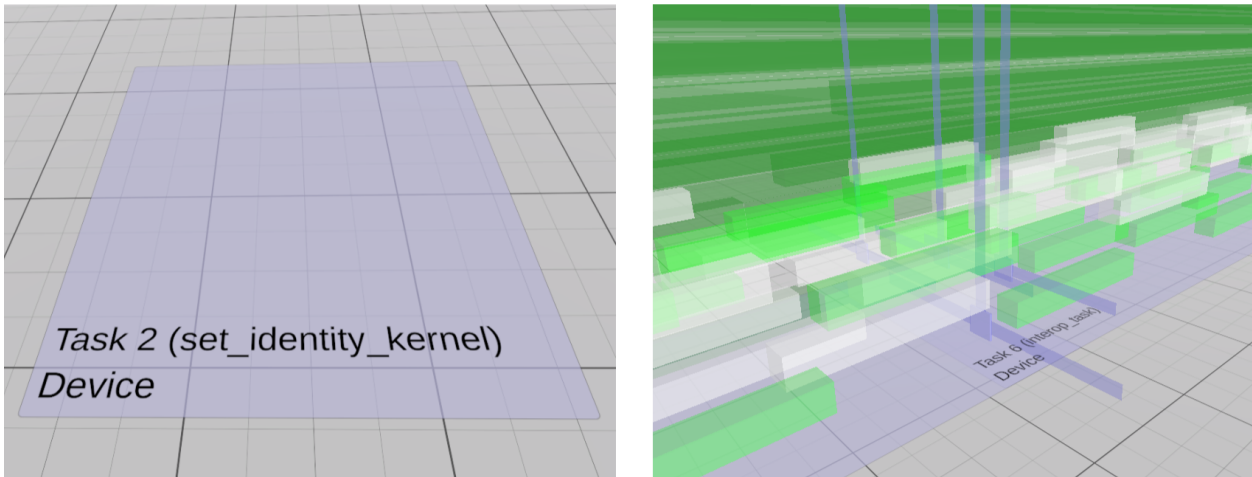


Figure 14: Auxiliary information displayed within the scene. On the left: An example of a Celerity task shown as a flat "panel" on the ground, containing the tasks description and scaled to span the extent of the task. On the right: Horizon-type commands with special highlighting to make them easy to pick out in a cluttered timeline.

listing 2. Since the critical path does not change during the runtime of the visualizer (it only depends on the trace data, which is static), it is only computed once and subsequently cached for all future calls while that trace file is loaded.

4.3.4 Auxiliary Information

In addition to the command-specific information detailed in the previous sections, more contextual information is displayed in the scene. This includes Celerity tasks (not to be confused by commands of the *type* task), which can be thought of as individual sections of the overall execution. In the context of the visualizer they act as a navigation guide to the user when trying to find specific sections of the trace, or for inspecting the composition of a single task. As mentioned in section 4.2.1, the trace data does not contain explicit extents for the tasks. Instead, their duration is inferred from the commands which are a part of it, with the start time of the earliest command marking the beginning, and similarly the end time of the latest one the end. Tasks are visualized using flat "panels" on the ground, scaled and positioned to span the duration of the respective tasks. They also contain text describing the tasks and provide information such as their ids, names, and execution target devices. An example of a task is shown on the left side of figure 14. It should be noted that tasks are only shown in the linear layout mode. For the circular layout the information is entirely omitted in order to avoid visual clutter.

Another feature to help with navigating the timeline is the special highlighting on horizon-type command nodes. Horizon commands in Celerity act as a synchronization point for all compute nodes during execution. Therefore they tend to appear as separators between distinct parts of the computation, making them a valuable navigation aid when inspecting the trace. As is visible in the example given on the right side of figure 14, the highlighting makes horizon commands visually distinct and easy to pick out in a cluttered timeline.

4.3.5 Display Modes

In order to avoid visual clutter, a number of visualization elements are separated into individual display modes. Each of these modes also serves to highlight different aspects of the trace file, each of which might be relevant to different use cases. For example, if the user is interested in assessing the optimization of the generated timeline, the critical path and idle sections modes should be useful. Similarly, if the user is only interested in the types of data transfers that were performed, then the filter mode will allow them to focus on only the types of commands that are relevant to their goal. The rest of this section describes the available display modes and possible combinations thereof in detail.

Type Filter

The type filter mode limits the timeline to only show specific types of commands. Since the density of command nodes on the timeline tends to be high in some parts of the trace, being able to hide part of the timeline that is irrelevant to the current goal can be useful. Commands of the excluded types are hidden for the time being, with their game object being disabled. This has the added benefit of reducing the workload for rendering of the scene (this is tested in section 5.3). The mode is built to work seamlessly with the chunking system. If the range filter mode (i.e., the mode where the chunking system is enabled) is active, the type filter is only applied to the command nodes contained in the currently shown chunks.

Internally, the type filter is processed by using the `CommandType` enum as a bitfield mask, the structure of which is described in section 4.2.3. This mask is accepted by all other relevant methods in the visualization setup, which can efficiently test against it using binary operations. A typical example of such an operation is included in the code shown in listing 1 on line 19.

Recursive Dependencies

Since the timeline contains an overwhelming amount of dependency information, it makes sense to only show a subset of all dependencies at a time. In the recursive dependencies mode, only the ones relevant to a selected command are shown. As described in section 4.3.3, this includes the predecessor and successor dependencies of the command, as well as the ones part of the commands on the other side of those relations. The result is a series of dependency lines spreading outwards from the selected command node, with a number (called the depth) of other commands along either way. Figure 15 shows an example of this mode in action.

This mode allows the user to inspect the dependency arrangement surrounding a specific command, seeing which commands are its predecessors and successors, and following the resulting dependency chains. The amount of information visible can be adjusted using the depth parameter, effectively limiting the extent within which the dependencies are shown. It also offers an option to hide all command nodes not part of the highlighted dependencies. This is especially useful in densely packed sections of a timeline.

At the same time, the list menu is updated to show the dependencies as a textual description, similarly the buffer access information present in the default visualization mode. The list entries include the type of dependency, the id of the other command, and the execution start/end time difference between the two commands. Clicking on an entry in the list jumps to the command node on the other side of the relation, allowing for very fast traversal of the dependency lines. For more information on the jumping behavior see section 4.4.1.

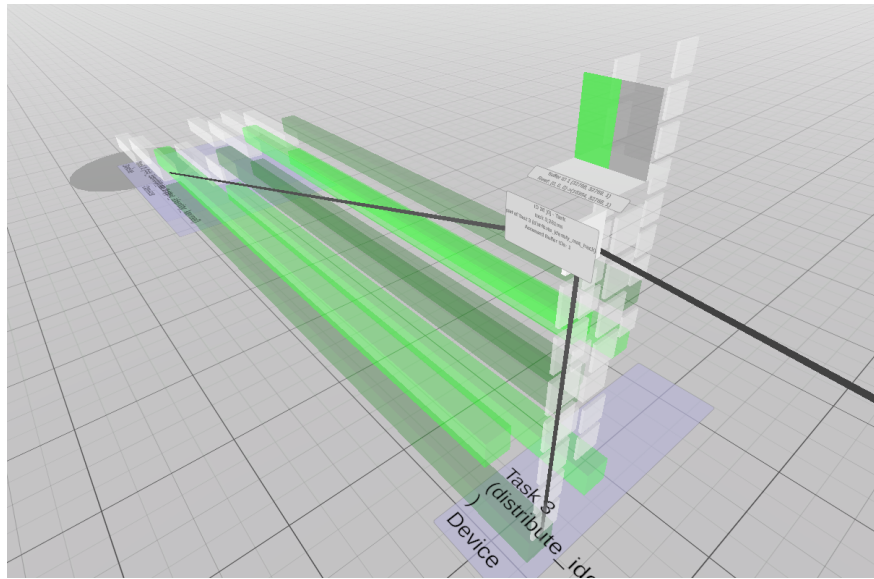


Figure 15: Example of the recursive dependencies visualization mode in action. The selected node in the center right has its command details shown. Dependency lines spread outwards between it and the other commands part of the relations. In this case the depth is set to one, so only the first "hop" is included.

Critical Path

As mentioned in section 4.3.3, the critical path represents the sequence of commands in the timeline taking the longest amount of time. In theory, reducing the length of the critical path directly reduces the overall execution time of the program, giving it an important role in assessing the optimization of the generated code.

The algorithm for computing the critical path returns a list of dependencies signifying the control flow along its length. These dependencies are shown using the established method of drawing lines between the relevant command nodes. In addition, the nodes along the path are all highlighted, meaning they are rendered as opaque, to make them stand out in the timeline. An example of the critical path mode is shown in figure 16. When this mode is active, the user can still manually select command nodes in the scene, and the command details are shown just as in the default visualization mode, allowing for inspection of specific nodes. However, the buffer accesses are no longer shown in the wrist menu list. The list is instead used to display all commands along the critical path in order from first to last. Once again, clicking on one of these entries jumps the timeline to the location of the respective command node. All of these features combined allow the user to easily assess the critical path of the trace, inspect the commands part of it, and have an overview of the dependencies making up the control flow.

However, this mode comes with some (current) limitations: Since the critical path passes through the entire timeline by definition, it contains commands which are part of virtually all chunks of the timeline. This prevents the chunking system to be active alongside it if the critical path should be visible in its entirety. It would be possible to adapt the system to only show the commands part of the critical path, or to only show the critical path within the chunks visible at the time. However, neither of these are implemented in the current state of the project since the integrity of the critical path as well as its context in the overall timeline are deemed to be useful. In the future these options could be implemented as options for the critical path mode.

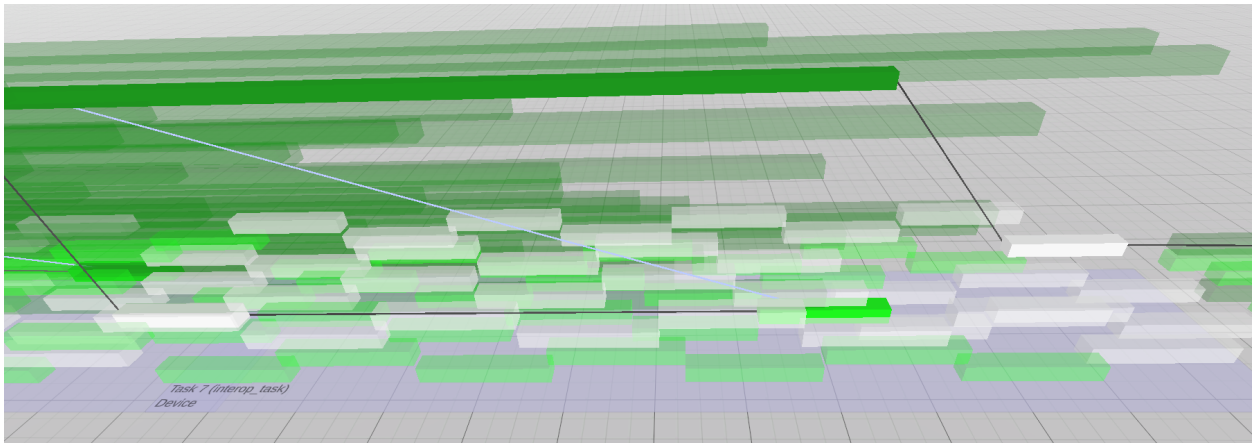


Figure 16: Example of the critical path mode being applied to a timeline. Following the path backwards (from right to left in this image), the path passes through a task-type command, onto a long-running await-push command, and then through a data dependency to its corresponding push command. This is a prime example for the potential utility of this mode since these kinds of inter-command relations are difficult to reconstruct by hand.

Idle Sections

Another way to determine the efficiency of the executed program is to look at idle sections, i.e. the times during which no meaningful computation was done. These sections can occur when, for example, the next computation task has to wait for necessary data to arrive from a different compute node, or when the compute node is waiting for a synchronization point. The goal in practice is to efficiently hide all the data transfer times by scheduling compute tasks in such a way that the data they require is already present and ready to use by the time the computation is meant to start. However, these efforts can (and frequently do) fail to completely hide the time taken by data transfers.

The idle sections display mode allows the user to easily identify these areas in the timeline. It comes in two variants: Command idle sections, meaning times during which no commands are active at all, and task idle sections, which shows all times during which no computations (in the form of task-type commands) are performed. Command idle sections allow the user to identify areas in the timeline where the program execution had to wait for external factors, such as a forced synchronization of all compute nodes. Task idle sections highlight areas where computation time is effectively wasted, hinting at potentially missed performance and therefore an opportunity to optimize.

To compute these sections all commands are first grouped by their compute nodes into separate lists and sorted by their start times. In the case of task idle sections, all non-task commands are left out of these lists. They are then each looped over once and the time difference between two consecutive commands' start/end times are computed. If the difference is larger than a minimum amount of time (set by the user through the interface) the detected section is added to a list of results. Once all sections are identified, they are visualized using colored and scaled boxes similar to command nodes, but visually distinctive as to stand out in the scene. They are positioned within the respective compute node's lane at the lower-most layer (see section 4.3.1 for more information on layers). Figure 17 shows an example of visualized sections in the timeline. At the same time all sections are listed in the wrist menu list, and once again the user can jump the timeline to a specific section by clicking on it in the menu.

4 Implementation

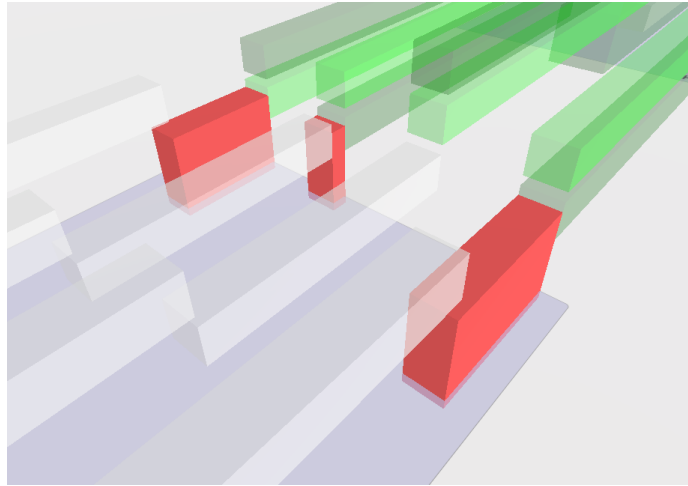


Figure 17: Idle sections shown on the timeline. In this case command idle sections are shown, meaning times in the trace during which no commands were active. The alternative mode is task idle sections, i.e. times during which no actual computation (in the form of task-type commands) was performed.

Range Filter

The range filter is the display mode making use of the chunking system described in section 4.3.1. It allows the user to set how much of the timeline around the origin should be displayed at a given time, called the vicinity or range size. In this context the origin represents the center of the scene, which acts slightly differently for the two layout modes. This is explained in more detail in the introduction to section 4.3. Any time the timeline is moved by any method (dragging by the user, set through the timeline slider, or jumping) the set of shown chunks is updated to match the new position at the origin. Limited testing shows a significant performance improvement when using the chunking system in combination with larger timelines (see section 5.3). Additionally, it was found that the mode does not generally lead to a loss of spatial orientation in relation to the overall timeline, which was one of the initial concerns with the chunking system.

Combinations

And finally, some combinations of the previously described modes are possible. In general these function as one would expect by simply providing the functionality of the two modes at the same time. When a combination is active, both settings interfaces are shown in the wrist menu at the same time, offering the full functionality of both modes. Both mode selection buttons are also highlighted to show that the modes are being combined. If the user activates a mode which is incompatible with one or more of the already active modes, the previously selected ones are automatically disabled. This interface is shown in figure 19 and explained in section 4.4.2. The possible mode combinations, in no particular order, are:

Type Filter and Recursive Dependency Mode In this case only dependencies involving the selected command types are shown, allowing the user to focus on specific parts of the trace dependencies.

Type Filter and Idle Sections With this combination and the command idle sections variation selected, only the commands whose types match the set filter are considered for the sections.

Type and Range Filter Simply combines the two as one would expect.

4.4 Virtual Reality Interface

As mentioned in sections 3.2 and 3.4, VR technology comes in many variations. Luckily the most common consumer devices stick to similar hardware designs for both headsets and controllers, allowing the trace visualizer to maintain some form of cross-platform compatibility.

However, in contrast to other hardware platforms such as smart phones, there is no current consensus on how user interaction in VR should work. For example, and as mentioned in section 2.4, there is no standard way to handle player movement. Multiple common implementations such as point-to-teleport or joystick-based movement are widespread, but they each come with their own drawbacks and usability challenges. Especially in the face of potential user health risks like the motion sickness mentioned by Korkut et al. [7], decisions about the user interface have to be well thought-out.

The following sections highlight the interaction methods chosen for this project as well as the technical groundwork necessary for their implementation.

4.4.1 Scene Movement and Interaction

Since movement in VR is a contentious topic [3], the idea for this project is to design the interface such that the user has the *option* to move, but is *not forced* to do so to access all the features available. The way this is achieved is by providing robust dragging and scaling input methods, allowing the user to move the timeline around their own position in the scene. This allows them to navigate the entire trace data without moving through scene-space themselves. In practice this method works surprisingly well, with the closest analog experience being two-finger touch inputs on mobile devices (see section 4.4.1 for more details).

At the same time the user is given the option to move around the scene using the two joysticks on the controllers, called twin-stick-movement. Boletsis et al. [3] concluded that while the joystick-based movement is not the best in regards to avoiding motion sickness, it is fairly easy to use once accustomed to it and "provided an overall experience of good quality". Additionally, a usable implementation is provided with the XR Interaction Toolkit, allowing for fast integration into the project.

Trace Parent System and Timeline Movement

Moving the entire timeline within the scene is done using the Unity object hierarchy. As mentioned in section 4.3, the entire visualization system is parented to the aptly named TraceParent. With this setup, the entire timeline can be moved by setting the position of that object. Its scale is also inherited by all visualization elements, allowing the timeline to be scaled using the same system. It should be noted that extra care has to be taken when implementing the individual subsystems in order for them to work with these transformations. All coordinate computations such as relative offsets (e.g. dependency line start/end positions) have to be computed either entirely in local space, or the results have to be adjusted to compensate for the global transformations (e.g. inverse scaling). An example of this can be seen in listing 4 in the case of axis-restricted scene components where a vertical offset has to be scaled inversely to the TraceParent's respective scale factor.

Internally, the system for moving the timeline works based on a given percentage along the timeline (i.e. 0% is the beginning, 100% the end). This acts as a uniform interface for most other systems which control the position of the timeline. In order to convert a given timestamp on the timeline to its relative percentage it is

4 Implementation

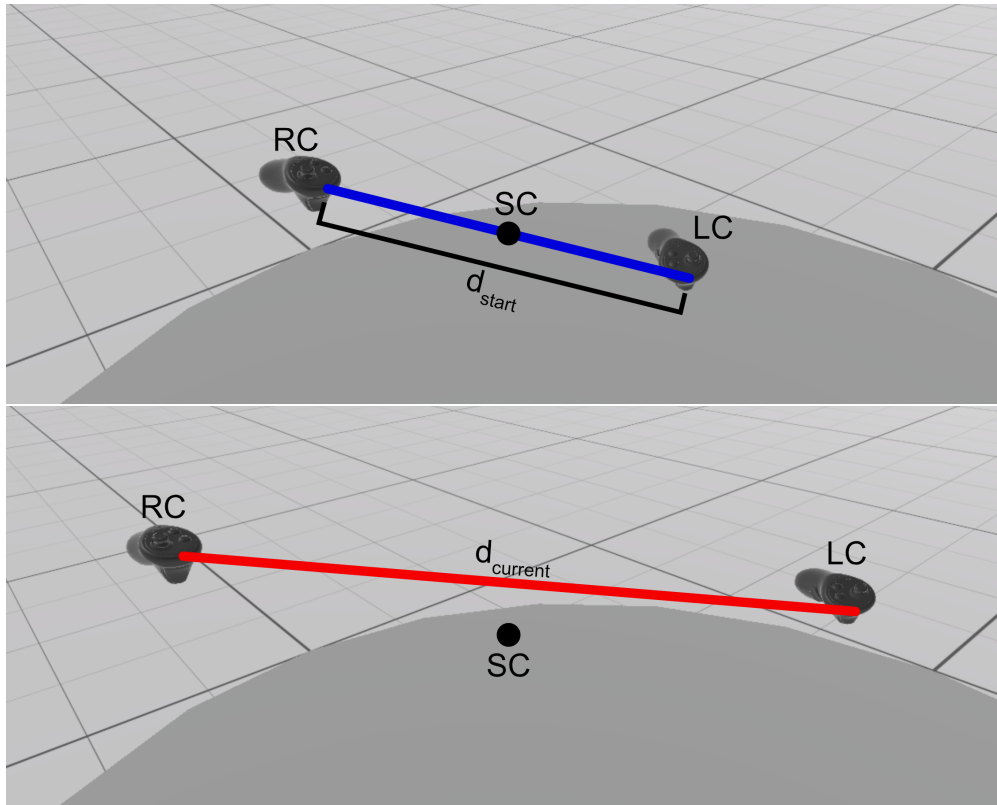


Figure 18: Geometric arrangement when scaling the timeline. Top: The starting configuration; The half-way point SC between the two controllers (RC and LC) becomes the scaling center for the operation, with the initial distance d_{start} being the reference distance. SC is static during the entire operation. Bottom: Configuration during scaling; The ratio of the current distance between the two controllers $d_{current}$ to the initial distance $\frac{d_{current}}{d_{start}}$ determines the scale factor by which the timeline is scaled around the point SC .

first converted to be relative to the starting time of the very first command in the timeline, and then divided by the total duration of the timeline. This way of moving the timeline to a specific location is mostly intended for jumping to specific locations, such as when a command is selected in the wrist menu list, or when the timeline slider is moved manually. In all of these cases its sufficient to move the timeline along its primary axis depending on the selected layout type.

However, when the timeline is moved by dragging using the controllers, movement in full 3D space is required. This type of movement is handled independently of the percentage-based system and covered in the next section.

Dragging and Scaling

As mentioned before, the dragging and scaling controls resemble two-finger touch controls common on smartphones and other touch devices. A good analogy to keep in mind is the use of a map application on a smartphone: The map can be moved by dragging with one finger, and scaled by either pinching or expanding with two fingers. When mapped to VR each finger becomes one controller and therefore hand of the user. To


```

1 private void UpdateDoubleControllerDrag()
2 {
3     // scale amount is ratio between current distance between controllers
4     // and the starting distance between controllers
5     Vector3 startPointConnectionVec =
6         (_rightControllerStartPoint - _leftControllerStartPoint).Value;
7     float startDistance = startPointConnectionVec.magnitude;
8     float currentDistance =
9         (_rightControllerCurrentPoint - _leftControllerCurrentPoint).magnitude;
10    float scaleAmount = currentDistance / startDistance;
11
12    // scaling pivot is mid-point between starting positions
13    Vector3 scalePivot = _leftControllerStartPoint.Value +
14        (startPointConnectionVec / 2f);
15
16    // timeline origin is position of timeline at start of scaling op
17    Vector3 origin = _startingPosition;
18    Vector3 distToTimelineOrigin = origin - scalePivot;
19
20    // current target scale based on controller positions
21    Vector3 newScale = _startingScale * scaleAmount;
22    float relativeScale = newScale.x / _startingScale.x;
23
24    // compute timeline position based on current scale
25    // = scale pivot + scaled distance between pivot and timeline origin
26    Vector3 postScalePosition = scalePivot + distToTimelineOrigin * relativeScale;
27
28    // apply computed values to scene
29    UpdateScale(newScale);
30    UpdateTransformPositions(postScalePosition);
31
32    // notify wrist menu about new position value
33    WristMenuManager.Instance.UpdateTimelineSlider(true);
34    // notify mode menu about new position value
35    WristMenuModes.Instance.UpdateRangeFilterModeAfterMove();
36 }

```

Listing 3: Code for computing the scaling values when the user is "double-dragging". This is executed once per frame during the scaling operation, with the starting positions being set on the first frame.

continue the analogy, "touching the screen" is equivalent to pressing the trigger on one of the controllers. This moves the entire timeline in sync with that controller ("dragging" it), allowing the scene to be moved in all three directions. For an overview of VR controller layouts and the triggers used for these interactions please see section 3.2.

When both triggers are held at the same time ("double-dragged") the timeline is scaled up or down. The scaling factor is computed as the ratio of the distance between the controllers at any point during the operation compared to their distance at the beginning. Since a simple scaling of the timeline around the scene origin would be disorienting, the point around which the scaling is performed is the half-way point between the controllers at the start of the double-drag. The spatial configurations of this operation are visualized in figure 18, and a slightly simplified version of the code for computing the scaling values is shown in listing 3. When either one of the controller's triggers are released, the timeline scaling at that point is locked in and the scaling operation ends.

4 Implementation

```
1 private void UpdateTransformPositions(Vector3 newPosition)
2 {
3     TraceParent.position = newPosition;
4
5     // offset task parent to be at TaskParentVertPos
6     Transform taskParent = GetTaskParent();
7     float diff = TaskParentVertPos - newPosition.y;
8     diff *= 1f / TraceParent.localScale.y; //invert scaling of TraceParent
9     taskParent.localPosition = new(0, diff, 0);
10
11     // offset node id label parent to be at 0 vertical position
12     Transform nodeIdLabelParent = GetNodeIdLabelParent();
13     diff = -newPosition.y;
14     diff *= 1f / TraceParent.localScale.y; //invert scaling of TraceParent
15     nodeIdLabelParent.localPosition = new(0, diff, 0);
16 }
```

Listing 4: Code for setting a new timeline position and adjusting the vertical local positions of the task parent and compute node id ring parent transforms. This is done to effectively pin these two at a set vertical position in world space. Note how the offset has to be multiplied with the inverse of the TraceParent y-axis scale to compute the correct vertical offset in local space.

Axis-Restricted Components

There are two parts of the trace visualization which do not follow the full range of movement that the rest of the timeline experiences. These are the task visualization in the linear layout mode, and the compute node id ring in the circular mode. In both cases they are pinned to the horizontal plane while still following the overall scaling operations and horizontal translations of the timeline in order to maintain alignment with the rest of the elements. Identically to the other visualization elements they are both parented to the TraceParent in the scene. However, after every moving and scaling operation their vertical position has to be adjusted to align with their target position (flush on the ground for the node id ring, slightly off the ground for the tasks). During this adjustment the current scale of the TraceParent has to be inverted in the vertical direction since the target height is in world space coordinates (i.e. independent of the current scale), whereas their own positions are in local space. The resulting code is shown in listing 4.

Twin-Stick Locomotion

The Unity XR toolkit provides four pre-implemented movement modes: Continuous (i.e. "twin-stick"), Grab (similar to the timeline movement by "dragging", but applied to the whole scene), SnapTurn (identical to Continuous, except turning is done in 45° increments), and Teleportation (the user can point at a spot on the ground to be teleported to). As mentioned before the Continuous or "twin-stick" movement mode is chosen due to its relative ease of use and robust implementation. In this configuration the left-hand joystick moves the user in whichever direction the stick is pushed, and the right-hand one rotates the user counter-clockwise and clockwise when pushed left or right, respectively. It should be noted that the movement directions selected by the left joystick are relative *to the left controller*, as opposed to other reference frames such as the user's look direction or a scene-constant axis. Additionally, the user is able to move their body in real space to perform smaller adjustments or naturally turn towards a desired direction.

Interacting With Scene Components

Besides the movement of both the timeline as well as the user within the scene, another main user interaction is the selection of command nodes. The goal is to allow the user to "click" any node to select it, an action once again very similar to the common interactions on 2D touch-screens. This functionality is implemented using a selection sphere attached to the right controller. It contains a sphere collider component, allowing it to interact (not physically, only in a logical sense) with all command nodes, which in turn contain box colliders. When the user presses the trigger on the right controller, a collider intersection check is done between the sphere and all other colliders within the scene. This task is delegated to Unity's built-in physics engine, which is optimized for such operations. The result is a list of intersecting colliders, out of which the selected command node (if present) is retrieved. After successful identification the information is passed on to all systems requiring the currently selected node.

Controller Click Disambiguation

Since there are effectively two interactions on the same right controller trigger (dragging and selecting), a way to effectively differentiate both intentions is needed. Conceptually a click on a node should happen while the controller is stationary, while a drag should (at least initially) consist of a larger movement. However, since user actions in **VR** are physical movements in real space, there will always be some slight amount of movement, even during the short amount of time for a click. This is further amplified by potentially inconsistent or "jittery" tracking of the controllers in space due to hardware limitations. Therefore, a dragging "deadzone" is defined. When the user first presses the trigger, the initial controller position is saved. While the distance from this position is less than 1 centimeter, the action is counted as a click (with the intention to select a command node). Once the distance is greater than 1 cm, the timeline dragging interaction is initiated, or the scaling action in the case of the triggers on both controllers being held at once. This system provides a clear differentiation between the two inputs and therefore reduces the number of misinterpreted actions. One notable side effect however is the introduction of a slight "skip" when performing a drag, due to the action only starting after the 1cm is cleared and the timeline position effectively having to "catch up" to the controller position.

In addition to the two mentioned interactions, there is another input type performed with the same right trigger: Clicking on **UI** components (the left trigger lacks the node selection functionality, but still has the dragging and **UI** click interactions). To avoid accidental scene movements or node selections when attempting to interact with **UI** interfaces, an additional check is performed when first detecting a controller click: If the controller is pointed at a **UI** component, the other two interaction modes are disabled until the trigger is released.

4.4.2 Wrist Menu

The wrist menu, as the name implies, is permanently attached to the user's left wrist (tracked by the left controller). It contains a number of 2D elements that allow the user to control the scene through various options and shortcuts. When pointing at the menu with the right controller, an aiming "laser" (i.e., a solid colored beam) is shown in roughly the direction the user's thumb is pointing, based on the controller's rotation. This interaction method is roughly comparable to using a laser pointer during a presentation, and is

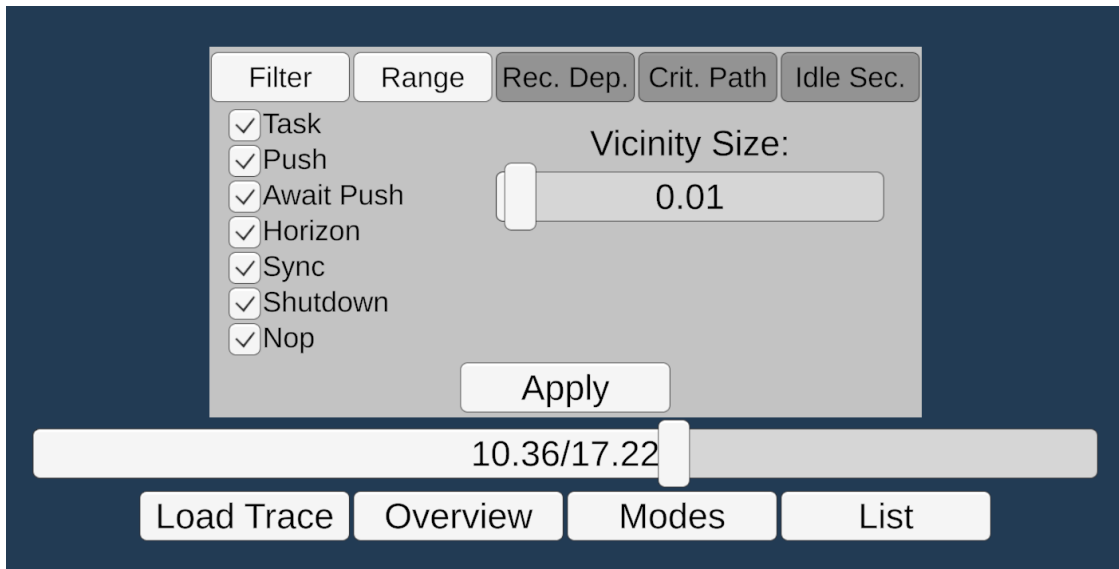


Figure 19: The wrist menu with the display modes category selected. The category selection buttons are visible along the bottom, with the timeline slider right above. At the top the display modes menu itself is visible with the filter and range modes currently selected. The settings for these modes are shown in the remainder of the panel.

well-established in VR with most major libraries and SDKs integrating it⁷⁸ (Unreal Engine 5 even integrates it in their VR Editor mode⁹).

The wrist menu itself is organized into four different categories: The timeline loading menu, timeline overview, display mode settings, and list menu. Figure 19 shows the buttons for switching between these categories along the bottom of the interface. The wrist menu additionally offers a high-level overview of the timeline by way of the timeline slider.

Timeline Slider

As shown in figure 19, the timeline slider is always visible in the wrist menu. It shows the current timestamp of the timeline at the scene origin, as well as the total duration of the trace. This information is continuously updated as the timeline is moved by any method. Additionally, the user can click or drag the slider to quickly move the timeline to a specific location. This feature is especially helpful for jumping between parts of the timeline, or to skip to a given timestamp.

Internally the slider is based on the UI slider component provided by the Unity engine. It's position along the length is set as a percentage value, which is computed similarly to the general timeline movement computation in section 4.4.1. When the user changes the slider's value directly, the new position is passed as a percentage value to the TraceMover component, which in turn sets the timeline position. The current position is also shown in seconds overlaid on top of the slider alongside the total duration of the timeline.

⁷<https://developers.meta.com/horizon/documentation/unity/unity-isdk-ray-interaction/>, accessed 03.02.2025

⁸<https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.1/manual/ui-setup.html>, accessed 03.02.2025

⁹https://dev.epicgames.com/documentation/en-us/unreal-engine/vr-mode-controls?application_version=5.0, accessed 03.02.2025

Trace Loading Menu

The first of the four main categories in the wrist menu is the trace loading menu, shown in appendix figure 27. Here the user can select a trace file to load and display in the scene. It also offers the option to switch between the linear and circular layout modes. The available trace files are read from the "traces" folder of the project files, and a manual refresh button to re-read the files is available in the menu. It should be noted that the names of the shown traces are only based on the file names; the content of the traces is not read at this stage for performance reasons. This has the side effect that no further information about the traces is visible to the user when selecting a file to load, which has the potential to lead to confusion. In the future the listed information could be extended by implementing a pre-scan of the trace files which only retrieves descriptive information from the traces (executable name, number of compute nodes, etc.) without parsing the entire data.

Timeline Overview

The timeline overview category shows general information about the currently loaded trace. At the moment this is limited to the run id for identification, the total number of compute nodes, commands, tasks, and buffers, as well as its total duration. The main purpose of these metrics is to allow the user to see some general statistics about the trace, as well as quickly identify which trace it is in case the file name is not descriptive enough. The shown information could be extended in the future with any additional content deemed relevant. Please see appendix figure 28 for an example of this section.

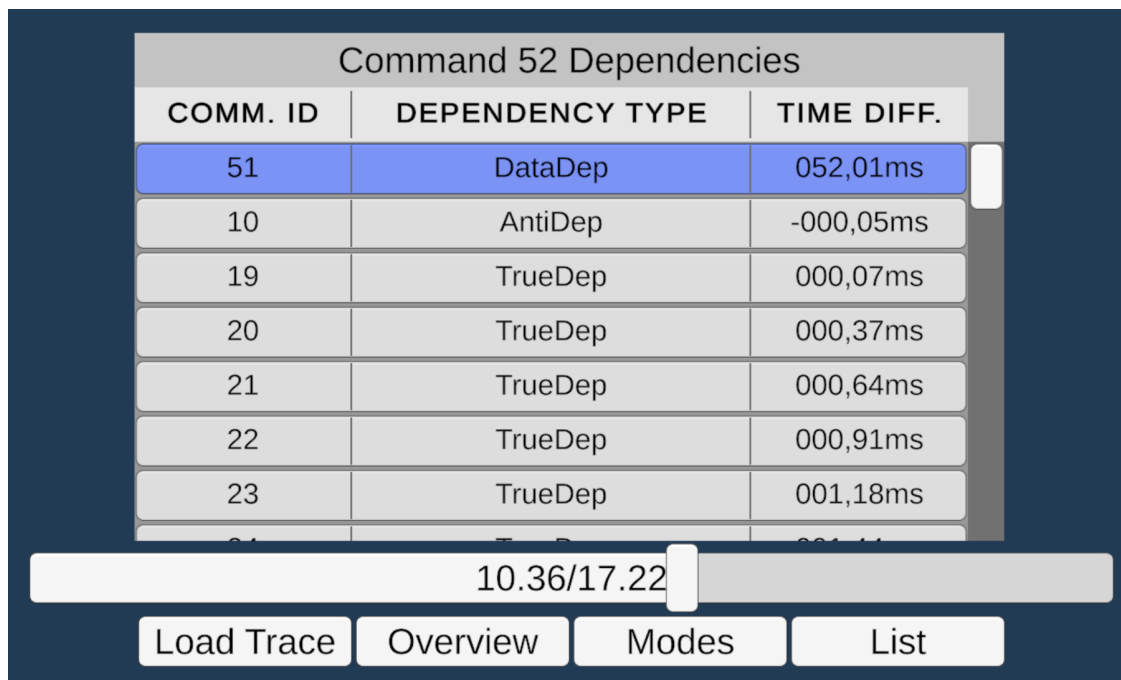
Display Modes Menu

The display modes (also called visualization modes) menu allows the user to change between the different visualization modes described in section 4.3.5. As visible in figure 19, each visualization mode has its own toggle button along the top of the panel. When a mode is enabled, the settings section in the main part of the panel is updated to show the appropriate options. Additionally, all other modes that are incompatible with the new selection are automatically disabled. This is communicated to the user in two ways: By graying out the respective mode toggle button, and by hiding its settings section in the panel. Furthermore, the settings for each of the modes are arranged in a way such that two conflicting modes tend to occupy the same space in the panel. Since mutually exclusive modes would therefore also be "competing" for the same visual space, it underlines the fact that they are also functionally exclusive. For a full overview of the different visualization mode settings please see appendix 2.

Since the application of a visualization mode can potentially lead to an initial performance impact (e.g. computing and showing the critical path), an explicit "Apply" button is provided. This allows the user to configure the visualization system to their liking before asking the system to apply it to the timeline. Another benefit of this two-step system is that the user is more prepared for a potential momentary decrease in performance (since they actively initiate the operation), something that can otherwise be jarring or disorienting in Virtual Reality.

List Menu

And finally as the last category, the list menu provides a way to display multiple types of lists. Its content is set to complement the currently selected visualization mode, acting as a quick overview in most cases. An example is shown in figure 20. Internally, the system utilizes the Unity engine UI components for the



COMM. ID	DEPENDENCY TYPE	TIME DIFF.
51	DataDep	052,01ms
10	AntiDep	-000,05ms
19	TrueDep	000,07ms
20	TrueDep	000,37ms
21	TrueDep	000,64ms
22	TrueDep	000,91ms
23	TrueDep	001,18ms

10.36/17.22

Load Trace Overview Modes List

Figure 20: Example of the list menu in action, in this case with a push-type command selected while the recursive dependencies visualization mode is active. The first dependency listed (with blue background) is the data dependency connecting the push-command to its receiving await push-command. Clicking on one of the listed dependencies moves the timeline to the corresponding command.

scroll-able rectangle, configured to match the size of the menu and to only expand vertically. The list menu manager provides a versatile interface for creating new list entries based on three strings (one for each column) and a delegate to be executed when the user clicks on the entry. A similar method for setting the column header strings is provided, and the list is automatically scaled vertically (along with a matching scroll bar) to fit an arbitrary number of entries. Additionally, the system provides a basic form of caching by saving the type as well as the title of the last shown list. These values are checked against the next list to be shown, and if they match exactly the list creation is skipped.

At the moment there are four different list modes available:

Buffer Accesses

Each buffer access of a selected command node is shown as an entry consisting of the id of the accessed buffer, range of accessed memory, and access type. Clicking on the entries does nothing. This mode is active when no other visualization modes (apart from the type filter) are enabled, i.e. in the default visualization mode.

Command Dependencies

With the recursive dependency mode active, all dependencies of a selected command node are shown in the list. The data dependencies are always shown on top with a distinct background color. All other dependencies are listed below those, sorted by the relative time offset of the other command involved in the relation. Each entry consists of the id of the other command, type of dependency, and relative time difference between the two. Clicking on one of the entries jumps the timeline to the center position of

the other command node, making use of the functions discussed in section 4.4.1. Figure 20 shows a list of this type being displayed.

Critical Path

In the critical path visualization mode all commands on the path are listed, sorted by their start times. Each has their unique command id, start time, and command type shown, and selecting an entry jumps the timeline to the respective node.

Idle Sections

With either one of the idle section visualization types (command or task) enabled, the list menu is updated to contain all of the identified sections, sorted by their respective start times. Each section is listed with its duration, start time, and the id of the compute node to which it belongs.

For more information on the individual visualization modes and their implementations please refer back to the sections 4.3.2 and 4.3.5.

5 Results

All of the visualization and user interface components together result in the product of this thesis: A working prototype for visualizing Celerity execution traces in Virtual Reality. It allows the user to select which trace file to be loaded, how it should be laid out in space, and then offers a range of different visualization modes and comprehension aides for the data contained in the trace. Furthermore, it aims to provide an intuitive user interface by sticking to tried and tested methods and industry standards where possible.

The goal of the project was to make use of the advantages VR offers, namely the third dimension and its potential for better visualizations, and apply it to improve the display of complex execution traces. In order to gauge the success of the project three categories are laid out: General usability, a comparison to 2D solutions, and performance. However, since no actual user studies were conducted for this thesis, the first two categories will be based purely on projections and initial testing. These findings are by no means an empirical answer to whether or not the project succeeded being a "better" trace visualizer. Furthermore, the results shown in the performance section should be taken as a rough guide to the *current* performance of the project. It is by no means indicative of what *could* be achieved with more focused optimization work being done. There are a number of improvements which could be implemented to drastically improve the performance of the project which were omitted for this thesis, on the one hand due to time constraints, and on the other hand due to the focus being on a proof-of-concept, not a final product. These potential improvements are highlighted in chapter 6 on future work.

5.1 Usability

All of the user interfaces and interaction points are based on established methods. As such, the usability of the whole system should be comparable to the sum of the individual solutions. While the trace visualizer is yet to be evaluated as a complete unit in a comprehensive user study, it is reasonable to assume that it offers a good degree of intuitiveness in its user interface. While some of the design choices are commonplace in other applications, such as the list menu with a list of choices and a scroll bar, others are either VR specific or had to be adapted to work with VR controls.

One of the most contentious user controls in use in the trace visualizer is the movement system within the scene. As highlighted by Boletsis et al. [3] in their 2019 study, there are a variety of competing standards and ideas for handling movement in virtual environments given the hardware available. The twin-stick movement controls offer a good trade-off between usability, intuitiveness, and ease of use, and are chosen for these reasons for this project. For more details on the use of these controls see section 4.4.1.

However, not all user interface components have been investigated in user studies. Most notably, the timeline dragging and scaling system described in section 4.4.1 is based on a transfer of a control scheme widespread in 2D interfaces. Dragging with one finger and scaling or zooming by moving two fingers further apart or closer together is ubiquitous in mobile devices, and used in applications from image viewing to map applications. The VR interface is designed to mimic this input method and therefore make use of the intuitive understanding

5 Results

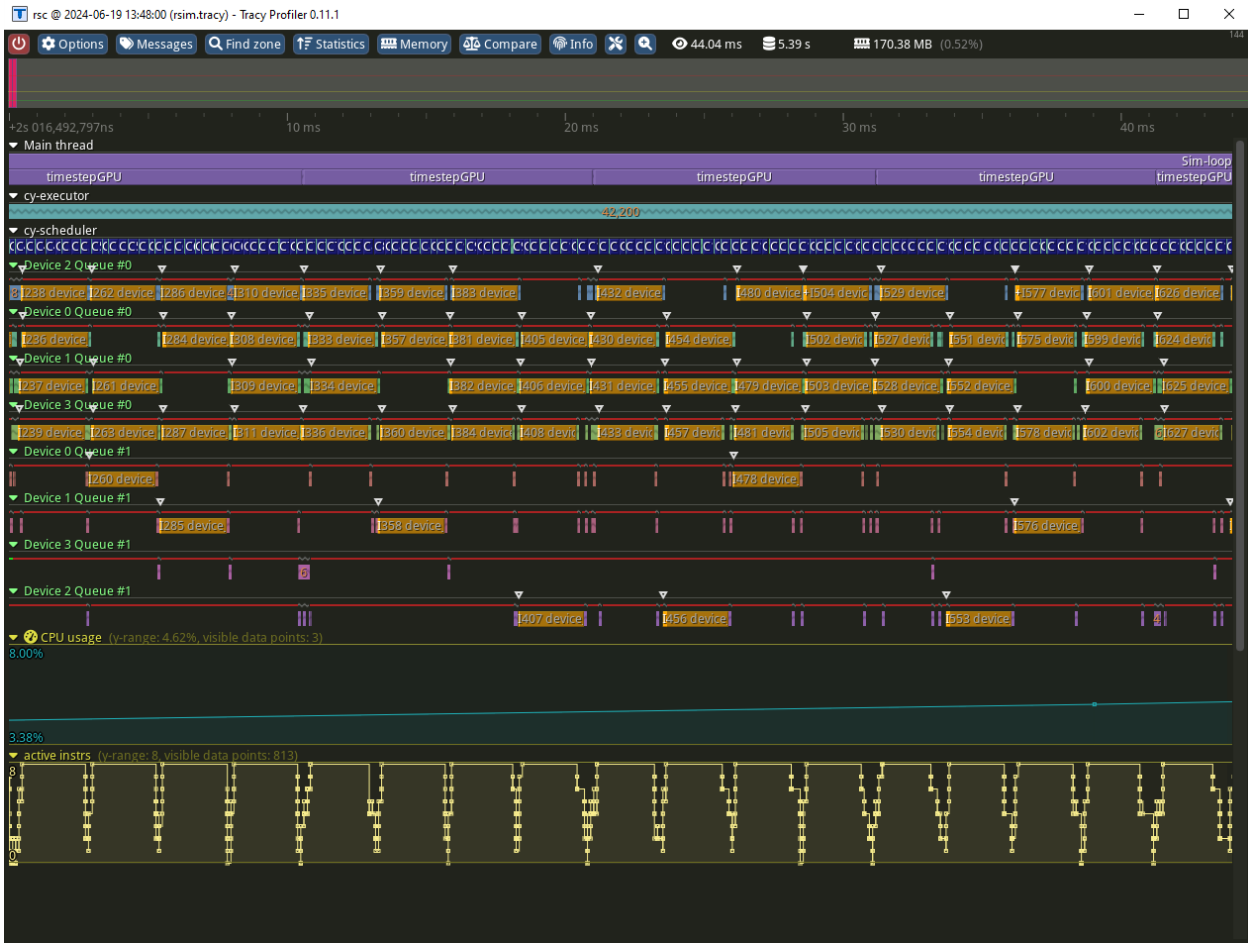


Figure 21: A Celerity execution trace visualized in the Tracy Profiler. The timeline is laid out similarly to the linear mode in the [VR](#) visualizer, with the commands being grouped by their worker nodes.

present from the 2D controls. However, mapping these transformations to 3D space is not completely trivial, as explored in the aforementioned section. Despite this, initial testing indicates that the [VR](#) version does have the same intuitive "feel" as its 2D counterpart.

5.2 Comparison to 2D Solution

The 2D trace viewer currently in use for the Celerity project is the Tracy Profiler¹. It is a feature-rich, open-source profiler, typically used in conjunction with specially instrumented code. However, in this use case it is instead employed to view the execution traces produced by Celerity, with figure 21 showing an example of the main window layout. Tracy is widely considered to be a state-of-the-art solution for trace visualization, supporting a variety of programming languages and operating modes, as well as offering good performance characteristics.

When comparing the interface of Tracy and the [VR](#) Celerity trace visualizer a clear similarity between Tracy's layout and the linear layout mode is apparent: Each compute node is shown as a separate "lane", arranged

¹<https://github.com/wolfpld/tracy>, accessed 04.02.2025

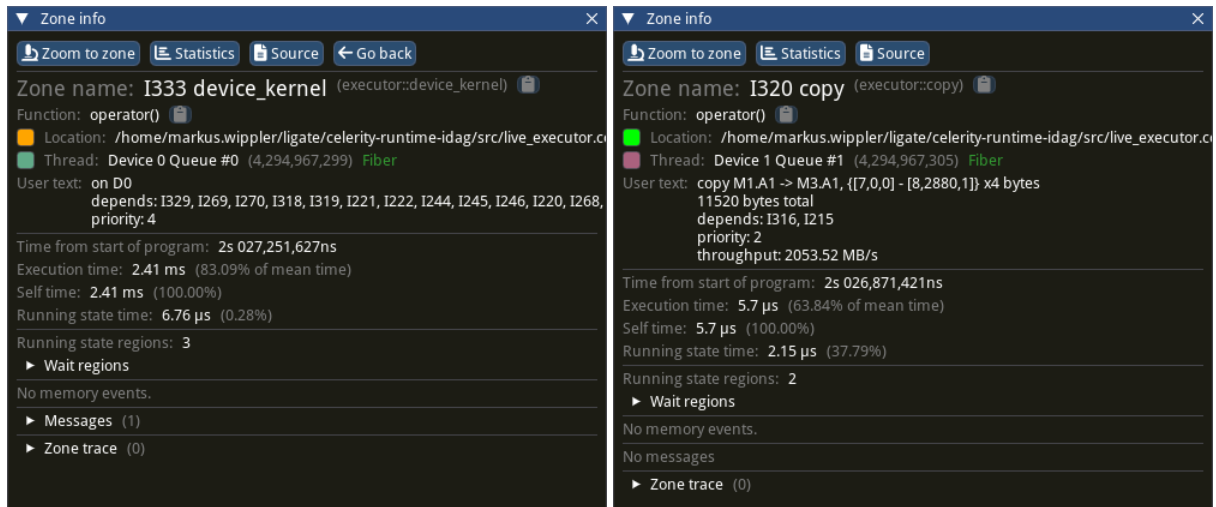


Figure 22: Detailed information on individual commands within Tracy. On the left: A task-type command with its dependencies shown as a list of ids; On the right: A push-type command, listing information about the transferred data and its destination.

vertically in Tracy and horizontally in VR. The individual commands are shown using differently colored bars (as opposed to the colored semi-transparent boxes), with their length signifying their execution time. Similarly to the grabbing and scaling implemented in the VR solution, Tracy allows the user to move the timeline and zoom in or out using the mouse.

In terms of functionality, visualization aides, and available data, there are a number of differences:

Detailed Command Information

When selecting a command on the timeline detailed information about it is displayed in both cases (see figure 22 for Tracy's version). However, the information in Tracy is purely textual in form, whereas the VR visualizer processes and integrates the information into the visualization. With the proper mode selected, dependencies are visualized using lines in space, arranged in an interactive list, and automatically highlighted on the timeline. In Tracy the user is left to manually search for the individual dependent commands using their ids, while also lacking any information on the dependency types or their immediate spatial arrangement (i.e., which ones in the list are "closer or further away"). Similarly, buffer access information is only shown in textual form (see the right side of figure 22), as opposed to the spatial representation present in its VR counterpart (see section 4.3.2).

Inferred Information

Tracy also lacks all of the information inferred during the VR visualizer's pre-processing step from section 4.2.2 - namely tasks, buffers, and data dependencies. As mentioned in that section, none of this information is explicitly stored with the trace data, and therefore isn't accessible to visualization tools like Tracy which only display the data present in the file. This results in the user having to manually infer and reconstruct these features themselves based on the information present, potentially hindering the speed at which the data can be interpreted. It also makes navigating the trace data more difficult since crucial information such as data dependencies (i.e., "where did this data come from?") is missing and no easy shortcuts like the jump-to-command feature (from section 4.3.5) are available.

Inspection Tools - Filtering and Critical Path

Since a Celerity trace can have a high density of information, especially in compute-heavy parts of the

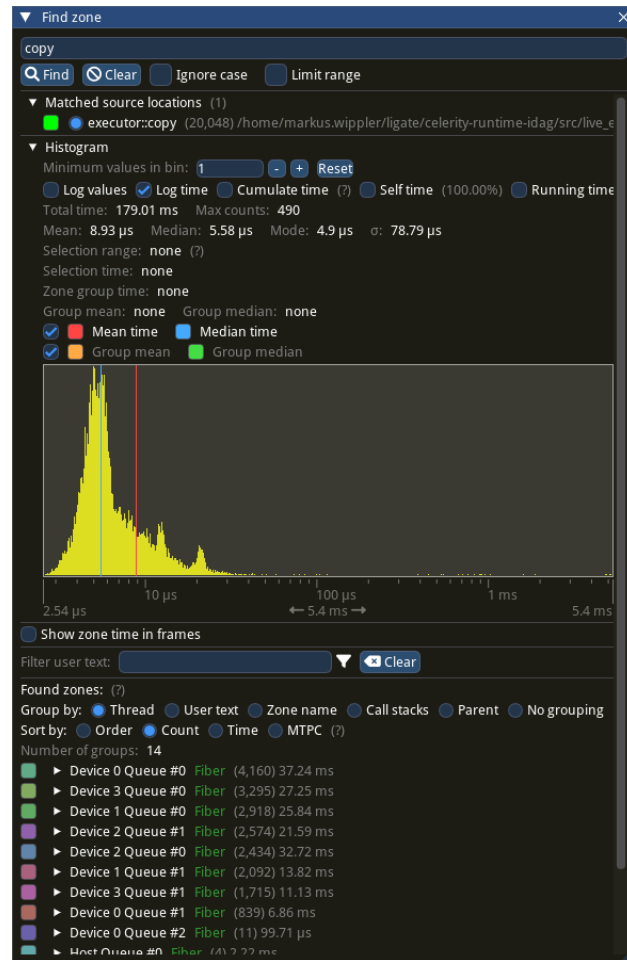


Figure 23: Example of the search function in Tracy. When searching for the keyword "copy", all push and await-push commands are shown as a density-histogram in the center, as well as grouped by their respective compute nodes below. This acts as the closest analog to the filter visualization mode from section 4.3.5.

execution, a way to filter out irrelevant data can be beneficial. While not directly offering an analogue to the filter mode (from section 4.3.5), Tracy does allow the user to search for various information within the data and displays an overview of the matching results, as seen in figure 23. This does offer a way to filter the trace data based on user-input keywords. However, as noted previously with other pre-computed data, Tracy has no way to compute the critical path and visualize it similar to the system in section 4.3.5. Whereas in previous cases like data dependencies the user could infer the data themselves on the fly, reconstructing the critical path is effectively impossible to do manually due to the potential length and complexity of the path.

Additional System Information

As opposed to the VR visualizer, Tracy includes detailed system information in the trace visualization. Figure 21 shows both a CPU usage graph as well as a running counter of the number of active instructions at any time alongside the bottom. While similar metrics could be implemented in the VR version it is missing at this time (in part since the old Celerity trace format does not store such information).

Name	File Size (KiB)	# Commands	Total # Deps.	Crit. Path Length	Duration (sec.)
Small	41.1	459	1315	15	32
Medium	153.8	1947	5227	15	31
Large	4623.1	49760	150186	521	2

Table 2: Key statistics of the three different execution traces used for the performance testing. The total number of dependencies includes both the command dependencies contained in the trace data as well as the inferred data dependencies. (Deps. = Dependencies, Crit. = Critical)

Performance and Usability

And finally, the major advantage that Tracy holds over its VR counterpart is its performance. Tracy is optimized to handle millions of events at the same time while offering a smooth user experience. It also uses far fewer hardware resources compared to the fully-3D rendered VR application for obvious reasons. Furthermore, it does not require the user to set up and don VR hardware before use, and supports a wide variety of conventional hardware platforms.

5.3 Performance

As mentioned before, the project as a whole is meant as a proof-of-concept and as such is not optimized for performance. Nevertheless, a number of implementation decisions were made with execution time and scaling in mind, especially as part of the more compute-heavy components of the application. The range filter and its associated chunking system (see section 4.3.5 and 4.3.1) as well as the type filter (from section 4.3.5) visualization modes are also designed as a way to improve the runtime performance of the application.

The performance of the trace visualizer is tested in two main ways: The execution times of critical parts of the application are measured as a way to quantify its scaling properties, and runtime FPS as a metric of overall usability. Please see appendix 3 for a detailed description of the hardware and software stack used for all of the following measurements. Table 2 shows a number of key statistics on the three different execution traces used for testing the application, which are from now on referred to by their relative sizes of small, medium, and large.

5.3.1 Execution Times of Selected Parts

Five different parts are selected and benchmarked independently in order to assess the performance and scalability of the application. The first three parts constitute the trace loading pipeline: Trace file deserialization, conversion of the loaded data, and creation of the command nodes of the timeline. Deserialization is done using the third-party C# Protobuf library *protobuf-net*². Quantifying its performance impact is helpful to exclude it as a bottleneck. The data conversion includes all the pre-processing detailed in section 4.2.2, which is heavily focused on data management. Command node creation consists of the processes from section 4.3.1, including node visuals creation and their placement on the timeline, as well as the creation of the chunking system data structure. The node creation is dependent on frequent interactions with the Unity engine for managing the visual components and their layout in the scene. The final two parts to benchmark, idle sections and critical path computation, are the computationally most expensive visualization modes to apply due to the amount of data they have to potentially consider during their execution. Overall, this selection of functions

²<https://github.com/protobuf-net/protobuf-net>, accessed 17.02.2025

5 Results

```
1 public static T MeasureExecTime<T>(string identifier, Func<T> func)
2 {
3     // invoke C# GC in order to prevent it running during the measurement
4     System.GC.Collect();
5
6     var watch = System.Diagnostics.Stopwatch.StartNew();
7     T res = func();
8     watch.Stop();
9
10    double elapsedMs = watch.Elapsed.TotalMilliseconds;
11    string timeString = string.Format("{0:0.00}ms", elapsedMs);
12    Debug.LogWarning("### Exec Time of " + identifier + ": " + timeString);
13    return res;
14 }
15
16 // Usage Example:
17 // Instead of
18 convertedTrace = TraceConverter.ConvertTrace(tracePayloadsList);
19 // Execute this
20 convertedTrace = MeasureExecTime<Trace>("trace conversion",
21     () => TraceConverter.ConvertTrace(tracePayloadsList));
```

Listing 5: Method used for measuring the execution time of a given function. As shown in the example in lines 20/21, it acts as a drop-in replacement for a regular call to the function to measure (line 18), taking its return type as a template parameter and passing the returned value along after the execution. Note the manual invocation of the C# garbage collector in line 4 before the function call in an effort to prevent it from running during the execution.

gives a decent overview of potential bottlenecks in the application. When combined with the three different trace sizes and their known properties from table 2 a rough estimate of program scalability can be made.

All measurements are performed using the code from listing 5. The MeasureExecTime function is usable as a drop-in replacement for any function calls as seen in the example on lines 16-21. In order to use it on functions with a *void* return type, an overloaded method without the template parameter and taking an *Action* instead of *Func<T>* is implemented (omitted from the listing). All timing is done using the System.Diagnostics Stopwatch³ included with the .NET runtime. It automatically uses hardware counters for high-accuracy measurements when available. Thanks to this, the reported resolution of the Stopwatch is 100 nanoseconds on the hardware used for all of the benchmarking. Another thing of note is the manual invocation of the C# garbage collector on line 4. When it is running in blocking mode the execution of all other code is temporarily paused until it is complete. Since the garbage collector is automatically invoked whenever the runtime system

³<https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.stopwatch?view=net-9.0>, accessed 17.02.2025

Name	Deserialization	Conversion	Com. Node Creation	Idle Sections	Critical Path
Small	83.29	12.99	28.02	3.92	1.90
Medium	92.01	17.41	99.12	4.48	2.00
Large	411.57	269.33	2384.71	20.87	4.35

Table 3: Average execution times for five functions for each of the trace sizes. A clear increase in runtime is apparent for the first three functions, which are dependent on the size of the trace. All measurements in milliseconds and as an average of five executions each.

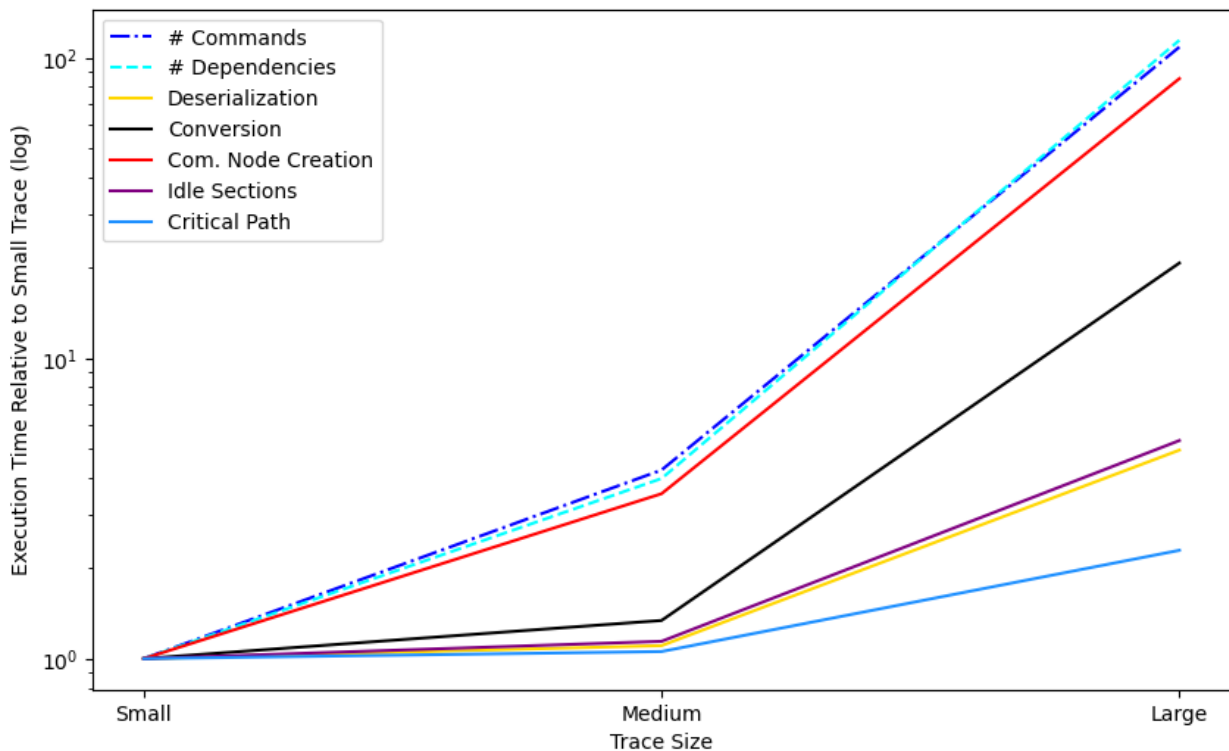


Figure 24: Relative execution times (solid lines) compared to trace size indicators (dashed lines), giving an idea of the program’s scaling properties. All values are factors relative to the respective times for the small trace (e.g. medium trace contains 4x as many commands compared to the small trace, but the idle section computation only takes 1.14x times as long). See appendix table 4 for the full values used in this chart.

deems necessary, there is a significant chance that it could run during the measurement of a function’s runtime, and as a result skewing the measured time. Therefore it is manually invoked and allowed to complete its collection cycle before the start of the measurement (on line 6) in the hopes that the runtime system does not invoke it again during the function execution. While there are methods to prevent garbage collection from running for short periods of time (such as `GC.TryStartNoGCRegion`⁴), it can lead to memory management issues, especially with a complex system like a game engine. As such the implementation used and shown in listing 5 relies solely on manual invocation with the results showing good stability between runs. The final results are shown in table 3, where each value is the mean of five executions each.

As expected, the execution times for each of the functions increases with larger trace files. However, the rate of that increase is very different between them. Comparing the execution times for the medium and large traces to the small trace and plotting those ratios yields figure 24 (note the logarithmic y axis). As a reference for the relative sizes of the traces, their number of commands and total number of dependencies is included in the graph as well. While the runtime for the command node creation (unsurprisingly) closely follows the number of commands in the trace file, the rest of the functions scale notably sub-linearly. Trace file conversion does take significantly more time for the largest trace size, however only ≈ 20 times as long, as compared to the ≈ 108 times as many commands present in that trace when compared to the smallest size. Most notably, the computation time for the critical path barely increases in relation to trace size. All ratios underlying the

⁴<https://learn.microsoft.com/en-us/dotnet/api/system.gc.trystartnogcregion?view=net-9.0>, accessed 17.02.2025

5 Results

figure can be found in table 4 in the appendix.

The observed scaling behavior depends on the respective function implementation and which parts of the trace data they interact with. In the case of command node creation, the same procedure of creating and positioning a new command node in the scene is performed for each command in the trace. This process largely depends on calls to the Unity engine, which tend to be somewhat computationally expensive. Similarly, the trace conversion process has to scan through the entire data multiple times to perform its pre-processing steps. However, in this case the larger trace file sizes likely allow for a more efficient computation due to predictable data access patterns and the resulting pre-fetching. The same is most likely the case for the idle section computation. These results also show that the *protobuf-net* library used for deserialization of the trace file scales well with increased file sizes. The most interesting results are the measured execution times of the critical path computation. They only slightly more than double from the medium to large trace despite the vastly greater number of commands in the trace and the significantly longer critical path (521 vs. 15 commands). Its computation relies entirely on the inter-command dependencies which form a graph spanning all commands (see section 4.2.2 for more information). Despite the large number of commands and dependencies present (and therefore possibly large separation between relevant commands in memory, potentially leading to pointer-chasing issues), the data structure and access patterns seem to scale very well with larger traces.

5.3.2 Runtime Frames Per Second

In order to assess the overall usability of the application its runtime performance is recorded while performing a standardized sequence of actions. To ensure repeatability of the experiment this sequence should be performed automatically and therefore identically each time. However, scripting movements in **VR** is nearly impossible due to the nature of the technology. Switching to non-**VR** rendering would defeat the purpose of the measurements since none of the overhead of rendering the scene twice (once per eye) and tracking data processing would be present. Since the main goal of this assessment is to gain a sense of the overall usability of the system given various trace sizes, the following alternative is chosen as the testing methodology:

The same sequence of events (load trace file, scale down timeline, move timeline to node, select node) is performed five times by a user in **VR**. Each run is done from a restart of the application and ends with a shutdown. Special care is taken to roughly match the timing of each of the actions, however slight variations are impossible to avoid with this setup. The recorded performance data is in the form of frame times, i.e. the time in milliseconds between two consecutive frames to be rendered (effectively "milliseconds per frame"). These frame times are converted to the more common unit of Frames per Second (FPS) and the results are slightly smoothed with a sliding window of 100ms to reduce some of the noise (see appendix figure 34 for a visual example of the smoothing effect).

Figure 25 shows an example of one such recorded sequence of actions. Each line represents one run, and while the overall length and timing is relatively consistent, key parts such as the trace loading (large downward spike at 3-4 sec.) and application shutdown (right side end of each line) are clearly offset for each one. This makes the computation of an average performance for each action in the sequence infeasible. However, the data as presented still gives a good indication of the overall usability of the system. In the example shown in figure 25 the target frame rate of 120 FPS is maintained most of the time, with the majority of lows still being above 100 FPS. In practice this results in a very usable experience with little to no noticeable performance issues.

In order to assess the impact of the two visualization modes intended to improve performance - the type filter and range filter modes - the same test is run with each of them enabled from the start. The type filter is set to

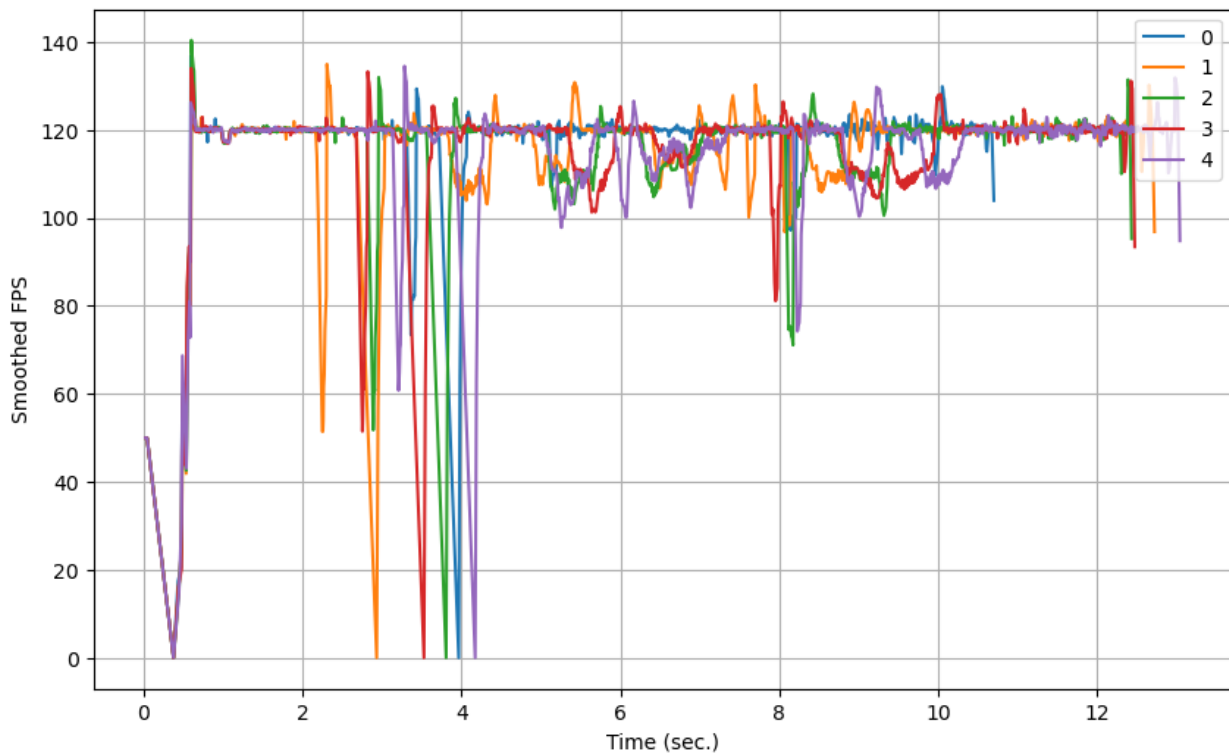


Figure 25: Recorded runtime performance using the medium size trace file. Each line represents a separate recording of a user performing the same sequence of actions. Distinct parts of the sequence are visible in each instance: Program start at 0.2 sec., trace load at ≈ 3 -4 sec., timeline manipulation from 5 to 10 sec., and shutdown at ≈ 13 sec. Note that all values are smoothed FPS, with a smoothing period of 100ms to improve visual clarity.

only display task-type command nodes, and the range filter is configured to show a 1 second vicinity around the origin. Since the performance of the medium size trace is relatively stable with the default mode enabled, the large trace is used to visualize the performance impact (the tests were still performed for the medium size trace and the results are included in the appendix section 4). As before, five runs are done for each configuration and the full results are included in appendix section 4. Figure 26 shows a direct comparison when performing the same sequence of actions on the large timeline with different mode configurations (for clarity only one representative run is shown for each one). While none of the runs could reasonably be referred to as having "stable" performance, both the type and the range filter modes show a significant improvement in FPS.

5.3.3 Future Optimizations

It should be noted that the Unity engine includes a profiler, however due to the high instrumentation overhead (leading to distorted measurements) as well as a lack of granularity in which parts of the program to measure, it was not used for any of the performance testing. It's main strength lie in providing a comprehensive view of the call hierarchy as well as the interactions with the engine itself, and less in accurately measuring runtime performance. One feature which is very useful in the context of optimization is its reporting of render pipeline metrics such as the amount of VRAM used or the number of drawcalls issued in a given frame. This

5 Results

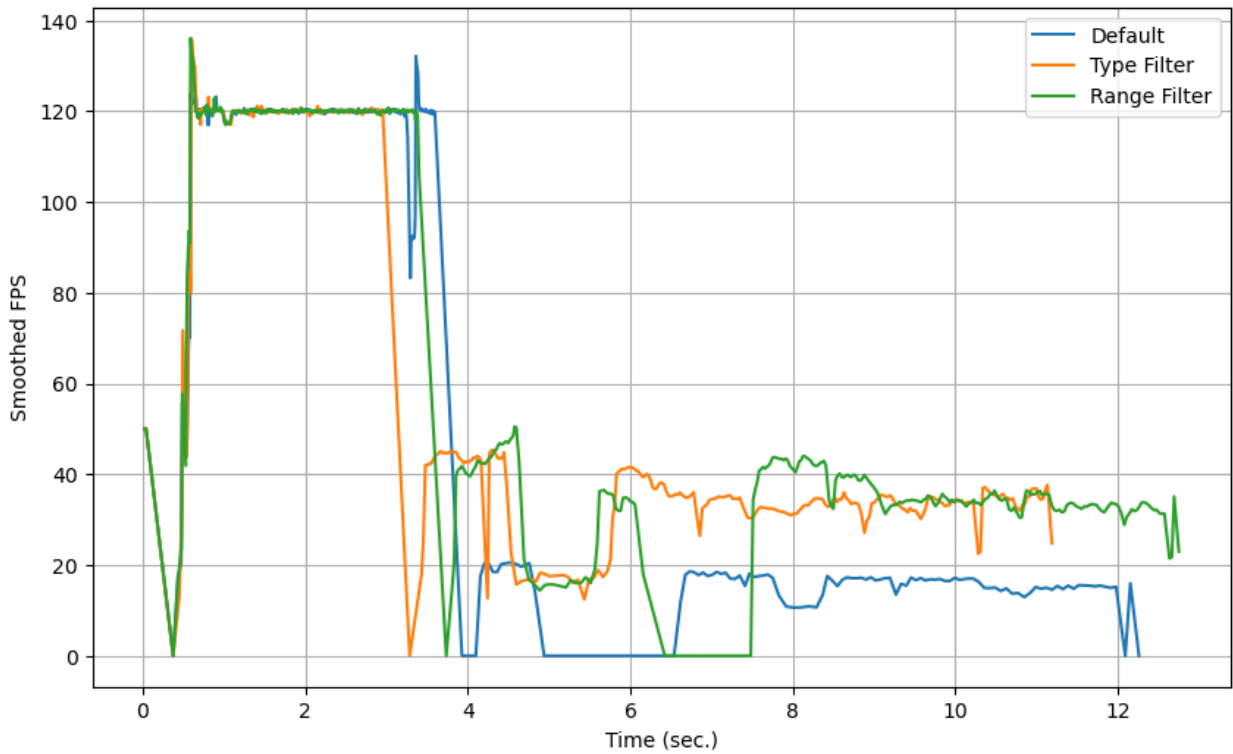


Figure 26: Comparison of runtime performance recordings using three different visualization mode configurations. In each case the same sequence of actions is performed by a user on the large-size trace file. The default mode has no special visualization modes enabled, type filter is set to only show task-type command nodes, and the range filter is set to show a 1 second long section around the origin. While none of the runtime performances can be considered usable in VR, both visualization modes show a significant improvement in FPS. Note that all values are smoothed FPS, with a smoothing period of 100ms to improve visual clarity.

information was instrumental in identifying the performance impact negated by using the pre-configured materials described in section 4.3.1.

To conclude, the current performance of the VR trace visualizer is serviceable for relatively small trace sizes. While there is an inherent higher computational overhead from running an application in VR as opposed to traditional 2D applications such as Tracy, visualizing trace data is still feasible using the technology. Given more work is invested in identifying and optimizing performance bottlenecks, trace files of sizes similar to the large one used for the testing should be able to run at the target frame rate of 120 FPS. One particular area for optimization is the rendering of command nodes in space: The current (relatively naive) implementation of using a dedicated GameObject, and therefore mesh and material, per node presents a major rendering pipeline bottleneck. Even though the optimization from section 4.3.1 reduced the number of drawcalls by roughly a factor of ten, the engine still issues more than logically necessary. An alternative would be to explicitly construct mesh objects at runtime to represent a number of command nodes and render these "clusters" instead. Interfacing more directly with the rendering pipeline would also grant much better control over the number and type of drawcalls issued, eliminating the somewhat opaque optimizations done by the Unity engine at runtime.

6 Conclusion and Future Work

The goal of this thesis was the development of a Virtual Reality Celerity trace visualizer with the intention of leveraging the advantages of the hardware platform to improve visual clarity and the intuitive understanding of the recorded data. Similar work has been done in the past in other application areas with promising results, especially in cases where the data to be visualized has a high degree of complexity and inter-dependence (see section 2.3). As such, a large part of the project was the design and implementation of various data visualization methods along with a user interface for controlling them.

Starting out with the execution trace file provided by a Celerity execution, the application parses the data and converts it into an internal data format. This step enables the program to be adapted to a different trace data format with relative ease in the future since the rest of the system is independent of the original file layout. Once converted the data is visualized in various ways: Two different overall layout schemes are implemented, a linear style similar to 2D solutions, and a novel circular layout. Additionally, a number of visualization modes aim to allow the user to focus on specific parts of the data, such as inter-command dependencies or idle sections. The system combines both traditional data representations (e.g. interactive lists) with VR specific visualization techniques, like the 3D representation of data buffers and their respective accesses. All of this is combined with a user interface designed to be intuitive and efficient to use. Since VR controls and interfaces are a potentially contentious topic, the aim was to use the most widely adopted or most user friendly solutions if they are available, and adapt tried and tested interface schemes from different fields otherwise. Additionally, all user interactions are designed to only utilize the most common hardware features present in most consumer-grade VR products in order to ensure broad hardware compatibility.

The resulting application compares favorably in terms of features against the current trace visualization tool used in conjunction with the Celerity project, namely Tracy. However, the VR visualizer is subject to both higher performance standards (to ensure user comfort during use) as well as a larger computational overhead due to the requirement of a full 3D rendering pipeline and stereo video output for two displays (one per eye). The current implementation, while functional with smaller trace sizes, suffers from significant performance issues with larger trace files, which are the more common use case of such a tool in practice. Nonetheless, the current implementation is sufficient for demonstrating that the visualization of complex trace data in Virtual Reality is feasible and has potential advantages compared to traditional techniques. Implementing an alternative method for rendering command nodes in the scene should significantly improve the application's performance characteristics and enable it to be used in practical applications.

The most important future development for the trace visualizer will be the adoption of the new Celerity trace format. This should be relatively easy to accomplish due to the decoupled design of the TraceConverter component. In order to allow the application to be used in conjunction with trace files produced by practical examples, its runtime performance should be optimized further. It has the potential to be especially useful in these cases since large traces tend to be more complex, in which case the additional visualization tools available could provide valuable help to the user. And finally, the usability of the program in its current form is solely based on projections from studies or respective application areas of the individual components. A comprehensive user study of the entire system would be useful for assessing the overall user friendliness and ease-of-use of the program, as well as to help identify parts of the interface in need of improvement.

Appendix

1 Wrist Menu Sections

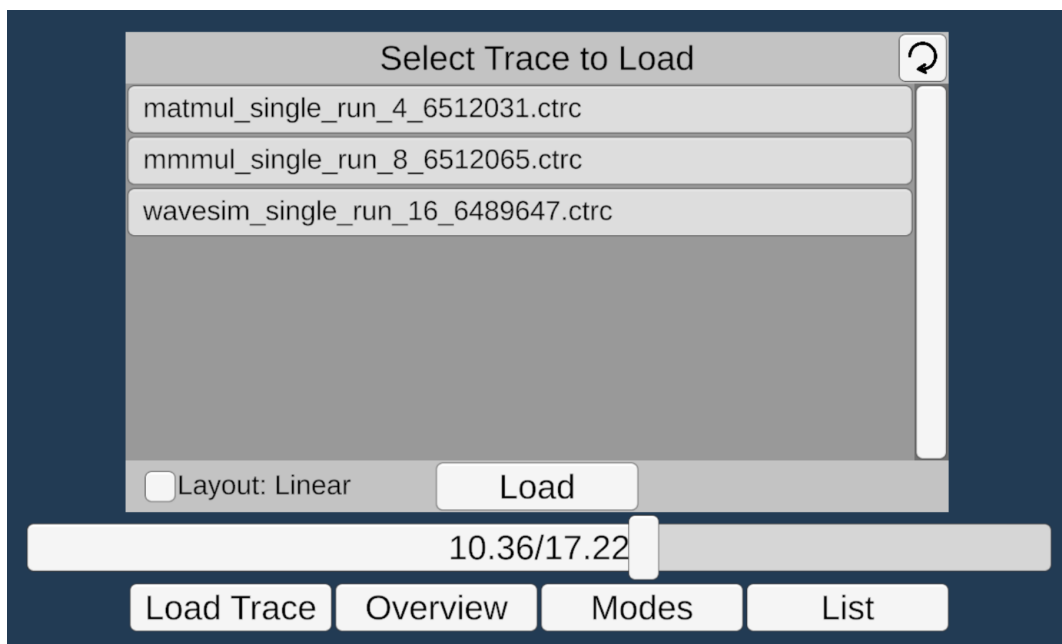


Figure 27: Wrist menu trace loading menu. All trace files found in the traces folder are listed and can be selected by the user. After selection the layout type is chosen using the toggle in the bottom left, and the trace is loaded using the "Load" button.

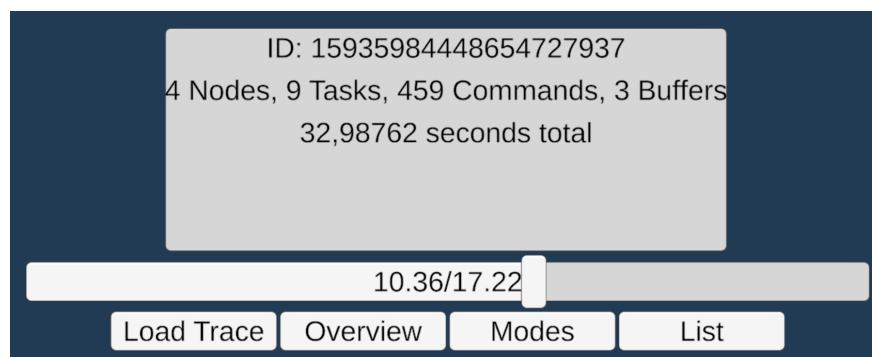


Figure 28: Wrist menu trace overview section. The interface offers five separate lines of text, the first three of which are currently populated with general statistics about the loaded trace file.

2 Display Mode Settings Interfaces

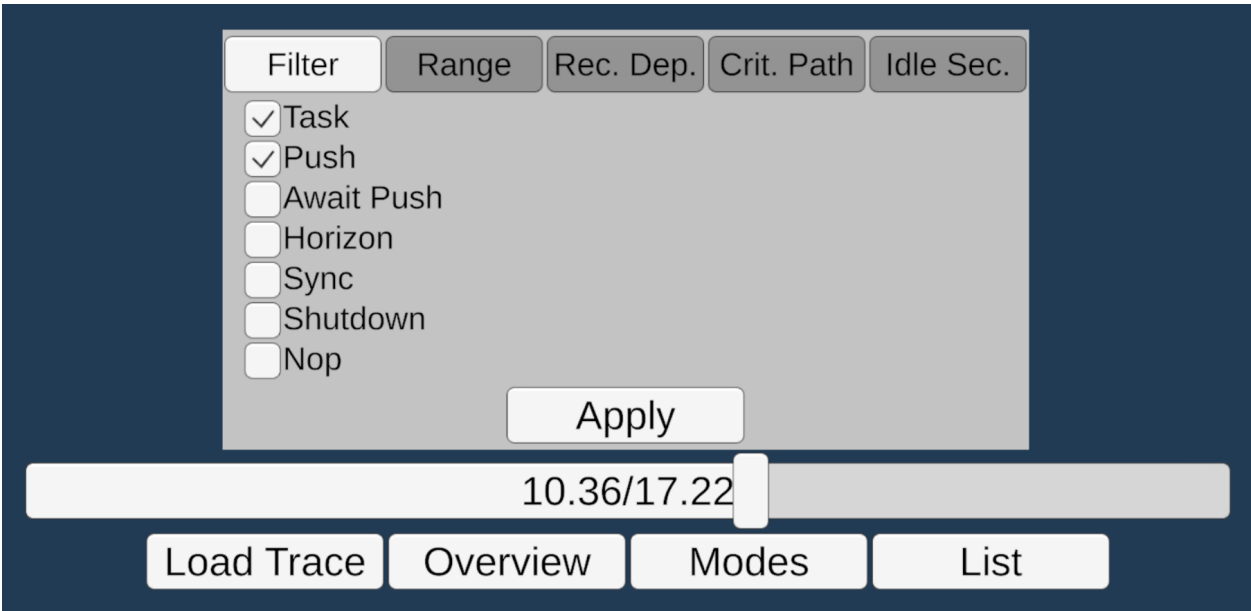


Figure 29: Modes settings menu with the filter mode selected. Each type of command can be enabled or disabled to be shown in the scene.

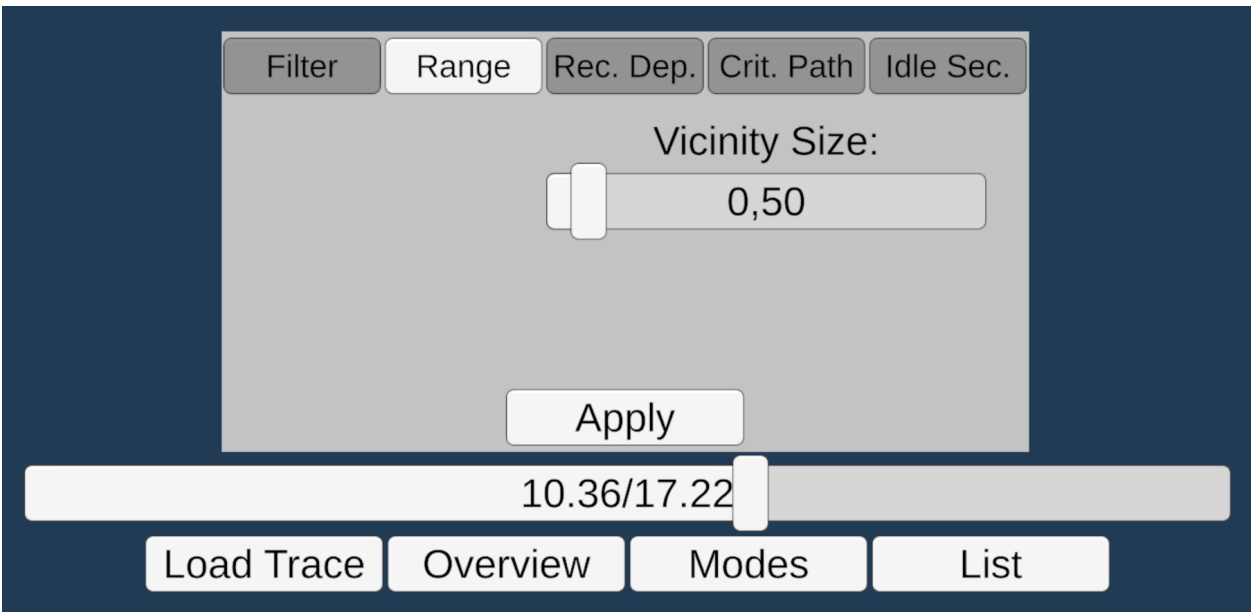


Figure 30: Modes settings menu for the range filter mode. The user can set the size of the timeline shown around the timeline origin using the slider labeled "Vicinity Size" in seconds.

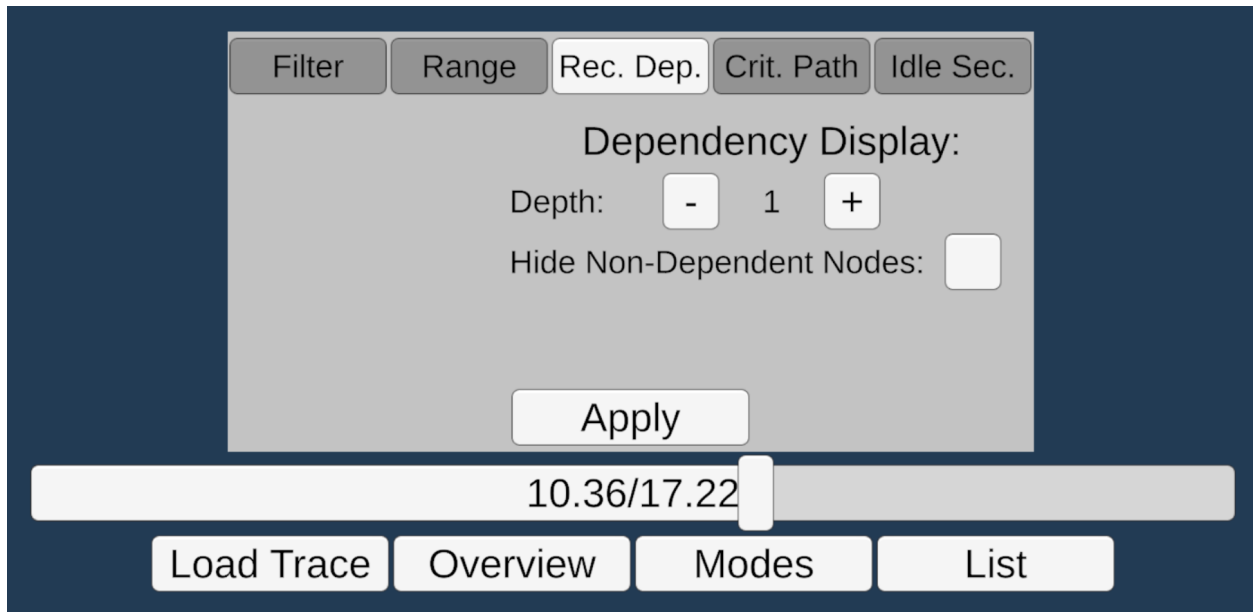


Figure 31: Modes settings menu for the recursive dependencies visualization mode. The depth parameter, changed with the "+" and "-" buttons, is the number of commands in either direction for which the dependencies should be shown. The setting below allows the user to hide all nodes not part of the dependencies found with the specified depth.

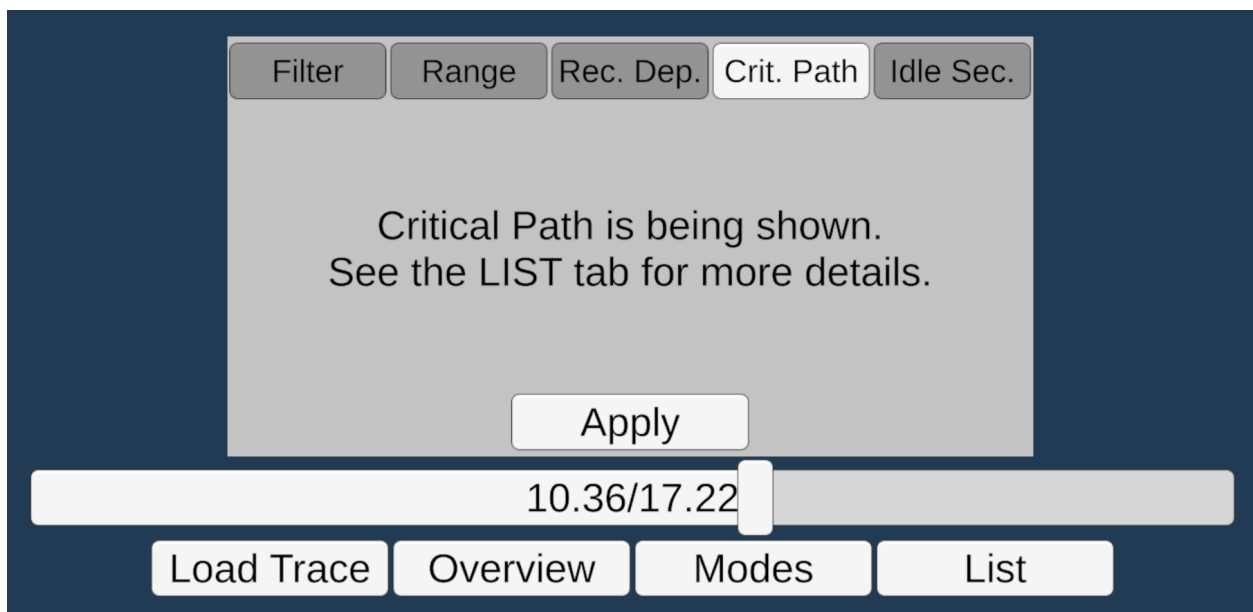


Figure 32: Modes settings menu with the critical path mode selected. This mode offers no further customization options.

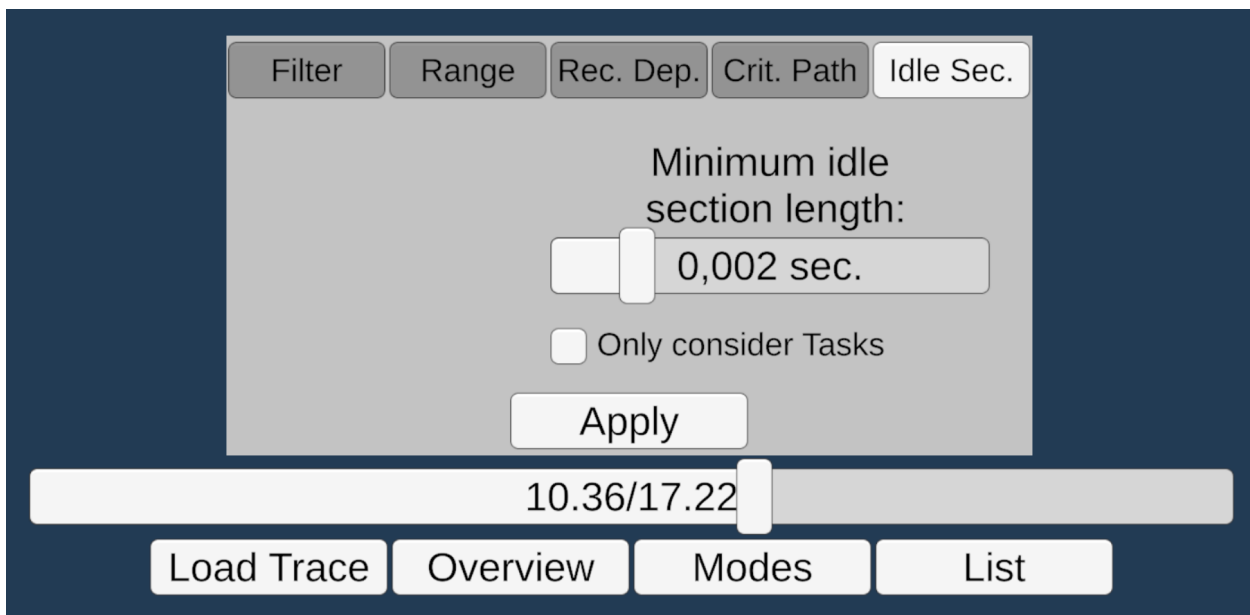


Figure 33: Modes settings menu for the idle sections visualization mode. The slider determines the minimum amount of time between two commands to be considered "idle", effectively controlling the sensitivity of the detection. The toggle below switches to the task-only idle sections mode.

3 Hardware and Software Used for Testing

The following hardware and software configuration was used for all performance-related testing:

- CPU: AMD Ryzen 7 5800X 8c/16t @4.7GHz
- RAM: 32GB 3200MHz DDR4
- GPU: Nvidia RTX 2080 with 8GB GDDR6 VRAM
- GPU driver version: 32.0.15.6636
- VR Headset: Valve Index with 2 x 1440x1600p displays (one per eye) @120Hz refresh rate
- Unity version: 2021.3.27f1
- Unity XR Interaction Toolkit version: 2.3.2
- OpenXR version: 1.7.0
- SteamVR version: 2.8.8

4 Additional Performance Measurement Results

Name	# Comm.	# Dep.	Deser.	Conv.	Com. Node C.	Idle Sec.	Crit. Path
Small	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Medium	4.24	3.97	1.10	1.34	3.54	1.14	1.06
Large	108.41	114.21	4.94	20.72	85.12	5.32	2.29

Table 4: Sizes (for # Commands and # Dependencies) and execution times (for remaining columns) relative to the small trace file. The values are based on the data shown in tables 2 and 3.

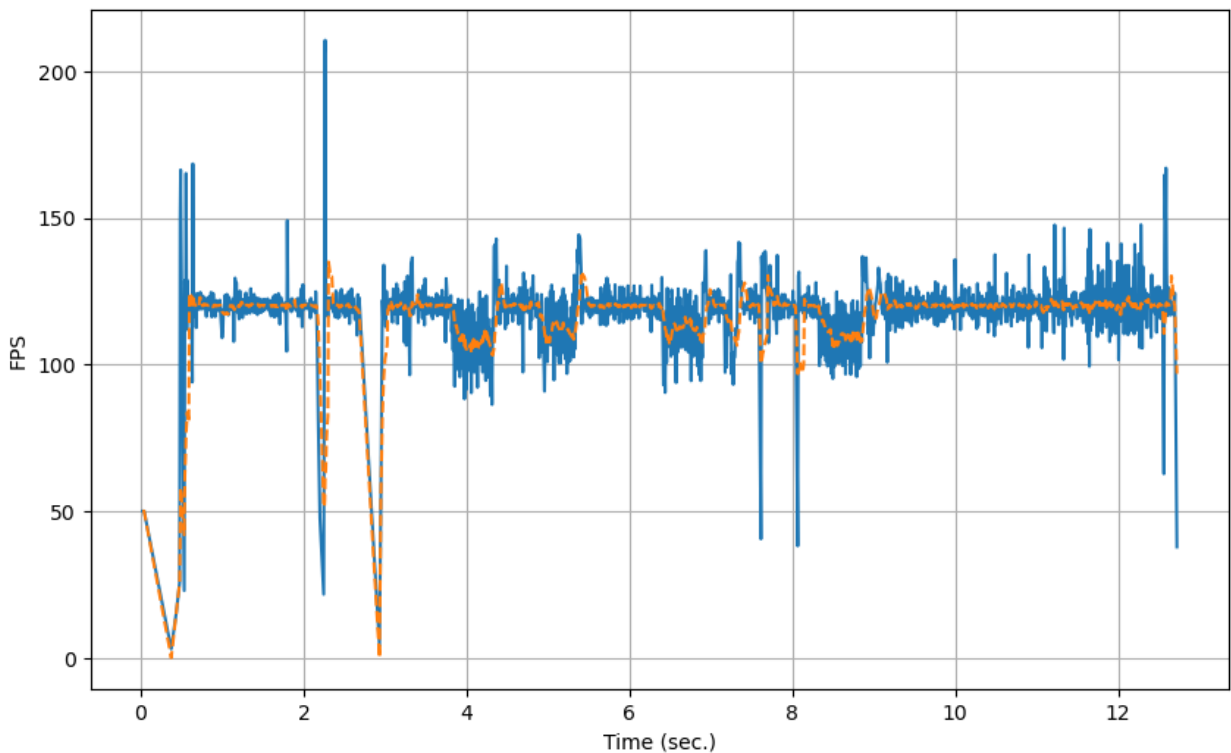


Figure 34: Visual example of the FPS smoothing applied to the raw performance data, in this case to a recorded execution using the medium size trace in the default visualization mode. While the raw data (solid line) has a lot of variation to it, the smoothed data (dashed line) offers a much clearer picture of the overall performance.

Glossary

AR Augmented Reality. 1, 6, 10, 15, 16, 67

DES Discrete-Event Simulation. 7

FPS frames per second. 14, 53, 56–58

HMD Head-Mounted Display. 6, 7, 9, 14, 15

HPC High Performance Computing. 6

IDE Integrated Development Environment. 4, 8

IVET In Vivo Exposure Therapy. 9

MR Mixed Reality. 15, 16, 67

SAD Social Anxiety Disorder. 9, 10

SPMD Single Program Multiple Data. 3

UI user interface. 17, 43–45

VR Virtual Reality. 1, 3, 5–11, 14–17, 39–41, 43, 44, 49–53, 56, 58, 59, 67

VRET Virtual Reality Exposure Therapy. 9

XR XR (umbrella term for **VR**, **MR**, and **AR**). 16, 17, 39, 42

Bibliography

- [1] Ikpe Justice Akpan and Murali Shanker. A comparative evaluation of the effectiveness of virtual reality, 3d visualization and 2d visual interactive simulation: an exploratory meta-analysis. In *Simulation: Transactions of the Society for Modeling and Simulation International*, volume 95(2), pages 145–170, 2019.
- [2] Narges Ashtari, Andrea Bunt, Joanna McGrenere, Michael Nebeling, and Parmit K. Chilana. Creating augmented and virtual reality applications: Current practices, challenges, and opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, page 1–13. Association for Computing Machinery, 2020.
- [3] Costas Boletsis and Jarl Erik Cedergren. Vr locomotion in the new era of virtual reality: An empirical comparison of prevalent techniques. In *Advances in Human-Computer Interaction*, pages 1–15. Hindawi, 2019.
- [4] Hubert Cecotti. Great paintings in fully immersive virtual reality. In *7th International Conference of the Immersive Learning Research Network (iLRN)*, pages 1–8, 2021.
- [5] Bas Cornelissen, Andy Zaidman, and Arie van Deursen. A controlled experiment for program comprehension through trace visualization. *IEEE Transactions of Software Engineering*, 37(3):341–355, 2011.
- [6] Toshiro Horigome, Shunya Kurokawa, Kyosuke Sawada, Shun Kudo, Kiko Shiga, Masaru Mimura, and Taishiro Kishimoto. Virtual reality exposure therapy for social anxiety disorder: a systematic review and meta-analysis. *Psychological Medicine*, 50(15):2487–2497, 2020.
- [7] Elif Hilal Korkut and Elif Surer. Visualization in virtual reality: a systematic review. *Virtual Reality*, 27: 1447–1480, 2023.
- [8] Troin McDonald, Will Usher, Nate Morrical, Attila Gyulassy, Steve Petruzza, Frederick Federer, Alessandra Angelucci, and Valerio Pascucci. Improving the Usability of Virtual Reality Neuron Tracing with Topological Elements. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):744–754, 2021.
- [9] Simone Romano, Nicola Capece, Ugo Erra, Giuseppe Scanniello, and Michele Lanza. On the use of virtual reality in software visualization: The case of the city metaphor. In *Information and Software Technology*, volume 114, pages 92–106. Elsevier B.V., 2019.
- [10] Philip Salzmann, Fabian Knorr, Peter Thoman, Philipp Schwandtner, Biagio Cosenza, and Thomas Fahringer. An asynchronous dataflow-driven execution model for distributed accelerator computing. *IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing*, pages 82–93, 2023.

Bibliography

- [11] Bhamidipati Satwik, Kalakoti Girija Manoj, Sai Rajendra Immadi, and Swaminathan Jayaraman. Visualization of distributed program execution. In *12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6. IEEE Computer Society, 2021.
- [12] Simon Su, Vince Perry, Luis Bravo, Sue Kase, Heather Roy, Katherine Cox, and Venkat R. Dasari. Virtual and augmented reality applications to support data analysis and assessment of science and engineering. *Computing in Science & Engineering*, 22(3):27–39, 2020.
- [13] Peter Thoman, Herbert Jordan, Philipp Gschwandtner, Thomas Fahringer, Biagio Cosenza, and Ben Juurlink. Celerity: Towards an effective programming interface for gpu clusters. In *26th Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, 2018.
- [14] Peter Thoman, Philip Salzmann, Biagio Cosenza, and Thomas Fahringer. Celerity: High-level c++ for accelerator clusters. In *Euro-Par 2019: Parallel Processing*, pages 291–303. Springer International Publishing, 2019.
- [15] Will Usher, Pavol Klacansky, Frederick Federer, Peer-Timo Bremer, Aaron Knoll, Jeff Yarch, Alessandra Angelucci, and Valerio Pascucci. A Virtual Reality Visualization Tool for Neuron Tracing. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):994–1003, 2018.